



西南交通大学
Southwest Jiaotong University

零售物品智能识别系统

基于 yolov8 的零售物品智能识别系统

设计人员： 2023112594——陶秋源
2023112476——马熙翔
2023112579——王昊宇
2023112567——程 赞
指导教师： 李威、殷成风

	文件标识	1.0
文件状态：	当前版本	V1.0
☒ 草 稿	拟稿人	王昊宇、程赞
☐ 正式发布	拟稿日期	2026 年 4 月 23 日
☐ 正在修改	审核人	陶秋源、马熙翔
	审核日期	2026 年 4 月 26 日

目 录

第 1 章 软件需求规格说明书	5
1 概述	5
1.1 背景	5
1.2 编写目标	5
1.3 相关术语定义	5
1.4 参考资料	6
2 总体要求	6
2.1 现状及痛点	6
2.2 系统目标	6
2.3 用户及角色分析	6
2.4 系统边界及上下文环境	7
3 功能性需求	7
3.1 主业务流程分析	7
3.2 功能用例分析	10
3.3 数据流分析	16
4 非功能性需求	18
4.1 性能需求	18
4.2 安全性需求	18
4.3 易用性需求	18
第 2 章 系统概要设计报告	20
1 引言	20
1.1 编写目的	20
1.2 项目背景	20
1.3 相关术语	20
1.4 参考文献	21
2 系统体系结构设计	21
2.1 系统特点分析	21
2.2 系统体系结构设计	22
3 接口设计	28
3.1 外部接口	28
3.2 内部接口	31
4 系统数据库设计	32
4.1 概念数据库设计	32

4.2 逻辑数据库设计	33
5 系统出错处理设计	38
5.1 出错信息	38
5.2 补救措施	39
第3章 系统详细设计报告	41
1 引言	41
1.1 编写目的	41
1.2 项目背景	41
1.3 参考文献	41
2 系统功能需求概要	41
3 智能视觉导购模块详细设计	42
3.1 功能模块概述	42
3.2 用户交互界面设计	42
3.3 类及接口细化设计	43
4 交易与结算模块详细设计	43
4.1 功能模块概述	43
4.2 用户交互界面设计	44
4.3 类及接口细化设计	44
5 后台商品与库存监控模块详细设计	45
5.1 功能模块概述	45
5.2 用户交互界面设计	45
5.3 类及接口细化设计	45
第4章 系统测试报告	47
1 项目简介	47
1.1 项目业务功能介绍	47
1.2 术语及主要名称介绍	48
1.3 参考文献	48
2 测试需求说明	48
2.1 编写目的	48
2.2 系统功能需求	48
2.3 非功能性需求指标	49
2.4 环境需求	49
2.5 测试人员要求	49
2.6 测试标准	49
3 测试计划	50
4 测试过程及用例	50

4.1 白盒测试用例	50
4.2 黑盒测试用例	51
4.3 性能测试	52
4.4 其他测试	52
5 测试报告及分析	53
5.1 测试报告	53
5.2 缺陷报告	53
5.3 分析总结	53
第 5 章 系统安装部署手册	54
1 编写目的	54
1.1 读者对象	54
1.2 参考资料	54
2 安装部署要求.....	54
2.1 服务器要求	54
2.2 支撑软件安装要求	55
2.3 应用软件部署要求	55
3 部署方案	55
3.1 环境部署	55
3.2 系统部署	56
第 6 章 系统用户操作手册	58
1 商超管理员角色操作手册	58
1.1 登录功能使用说明	58
1.2 商品管理功能使用说明	58
1.3 视觉调度功能使用说明	58
1.4 告警与看板功能使用说明	58
2 顾客角色操作手册	59
2.1 登录功能使用说明	59
2.2 查询功能使用说明	59
2.3 视觉识别寻物功能使用说明	59
2.4 购物车与结算功能使用说明	59
2.5 订单、支付与充值功能使用说明	59
3 系统后台角色操作手册	60
3.1 巡检任务运行说明	60
3.2 参数调整与手动触发说明	60
3.3 异常处理说明	60
3.4 运维建议	60

第 1 章 软件需求规格说明书

1 概述

1.1 背景

随着人工智能（AI）与计算机视觉（CV）技术的飞速发展，传统零售行业的数字化转型已成为必然趋势。传统商超主要依赖人工盘点与条码扫描结算，存在结算高峰期拥堵、用户寻物困难、库存数据更新滞后等显著痛点。本项目旨在利用先进的深度学习目标检测算法，结合 Web 全栈技术，开发一套低成本、高效率的“零售物品智能识别系统”，以期实现商超场景下的视觉导购、无感自助结算及供应链自动预警，全面提升运营效率与顾客体验。

1.2 编写目标

本文档旨在全面、准确地描述“零售物品智能识别系统”的功能性需求与非功能性需求。本文档将作为项目架构设计、代码编码、软件测试以及最终期末验收的核心基准文件，确保开发人员、测试人员与系统干系人对系统边界和业务逻辑达成一致理解。

1.3 相关术语定义

- **CV (Computer Vision):** 计算机视觉，一门研究如何使机器“看”的科学，本系统中主要用于处理摄像头视频流。
- **YOLO (You Only Look Once):** 一种端到端的实时目标检测（Object Detection）算法，本系统采用其轻量级版本实现商品的毫秒级定位与分类。
- **B/S 架构 (Browser/Server):** 浏览器/服务器模式，本系统的主要网络架构，用户只需通过浏览器即可访问系统，无需安装客户端。
- **FastAPI:** 一个现代、快速（高性能）的 Python Web 框架，基于标准 Python 类型提示，用于构建本系统的后端 API 接口。
- **ORM (Object-Relational Mapping):** 对象关系映射，本系统通过 SQLAlchemy 实现面向对象的数据库操作，避免直接拼接 SQL 语句引发的安全风险。

1.4 参考资料

1. 《软件工程：实践者的研究方法》
2. YOLO 官方技术文档与学术论文
3. FastAPI 与 SQLite 官方开发指南

2 总体要求

2.1 现状及痛点

- 顾客导向不足：在大型或陌生商超中，顾客往往难以快速定位所需商品，缺乏有效的室内视觉导航手段。
- 收银效率瓶颈：传统扫码结账需人工逐一寻找条形码，效率低下，高峰期极易造成排队拥堵。
- 库存管理滞后：依赖人工周期性盘点，无法做到数据的实时同步；缺货往往在顾客反馈后才被发现，导致销售机会流失。

2.2 系统目标

构建一个集“智能寻物、快捷收银、自动化库存管理”于一体的闭环平台。核心目标包括：

1. 实现毫秒级的商品视觉定位与识别。
2. 提供流畅的前后端交互，实现一键式购物车结算。
3. 建立基于阈值的智能库存预警机制，实现供应自动化流转。

2.3 用户及角色分析

- 顾客（Customer）：核心最终用户。主要需求为浏览在售商品、通过视觉系统定位商品位置、管理购物车及完成最终支付。
- 商超管理员（Admin）：后台运维主体。主要需求为录入商品基础信息（名称、单价、初始库存）、处理缺货告警、查看销售统计报表。
- 系统定时任务（System Cron）：自动化执行代理。在后台静默运行，负责在每次交易发生后校对库存水位，并自动向虚拟供应商派发补货工单。

2.4 系统边界及上下文环境

- 系统内边界：包含基于 HTML/JS 的前端交互界面、基于 Python FastAPI 的业务逻辑控制层、基于 YOLO 的视觉推理子进程、以及 SQLite 关系型数据库。
- 系统外边界：物理图像输入设备（USB 摄像头或监控探头）、操作系统的底层文件系统（用于存储模型权重）、模拟的第三方支付网关。

3 功能性需求

3.1 主业务流程分析

核心导购与结算业务分析（UC-01）

本业务流程涵盖了用户从进入系统到完成闭环的完整生命周期。

- 业务描述：
 1. 用户进入系统并验证身份。
 2. 用户浏览商品列表，可选择直接加入购物车，或触发“视觉定位”请求。
 3. 若触发定位，系统唤起本地摄像头子进程，利用 AI 算法在视频流中实时高亮目标商品（绘制 Bounding Box 与中心坐标）。
 4. 用户将所需商品全部加入购物车后发起结算。系统校验库存充足性，计算总价并生成订单。
 5. 结算成功后，系统并行触发库存核查机制；若库存低于设定阈值，则自动执行补货逻辑。
- 角色参与说明：
 - 顾客：业务的发起者与主要操作者，负责下达定位指令、确认购物车清单并支付。
 - AI 视觉模块：作为硬件交互执行者，响应顾客请求，提供毫秒级的商品视觉坐标。
 - 后端系统（FastAPI + SQLite）：作为业务大脑，负责拦截非法请求、执行库存扣减的原子操作，并判断是否触发下游链路。

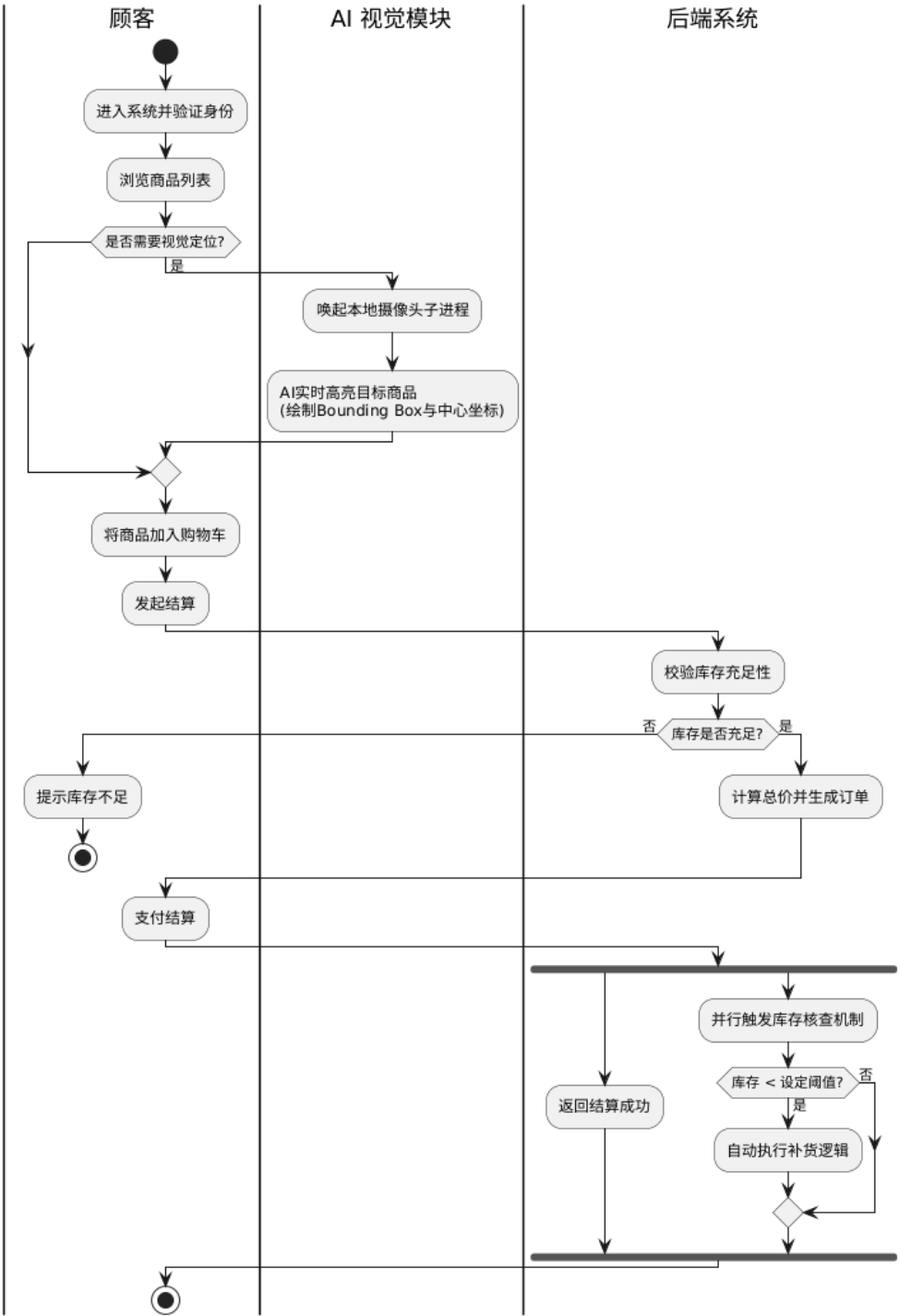


图 3-1

商品上架与基础数据维护业务分析（UC-04）

本业务流程主要描述商超管理员如何对系统底层的商品字典和库存容量进行初始化及日常维护。

• 业务描述：

1. 商超管理员登录系统后台管理面板。

2. 选择“商品录入/更新”功能模块。
 3. 输入商品的英文标识（需严格匹配 YOLO 模型的 names 标签）、零售单价以及入库数量。
 4. 后端系统接收到数据后，对数据库进行 Upsert（更新或插入）防重载校验。
 5. 如果数据库中已存在该商品，则将新入库数量与原有库存进行累加，并以最新填报的价格覆盖旧价格。
 6. 如果是全新商品，则在数据库中插入一条全新记录。
- 角色参与说明：
 - 商超管理员：数据的提供者，确保录入的商品标识与 AI 视觉模型训练集分类保持强一致性。
 - 后端系统：数据校验者，保证商品数据的唯一性，防止因重复录入导致数据库出现脏数据。

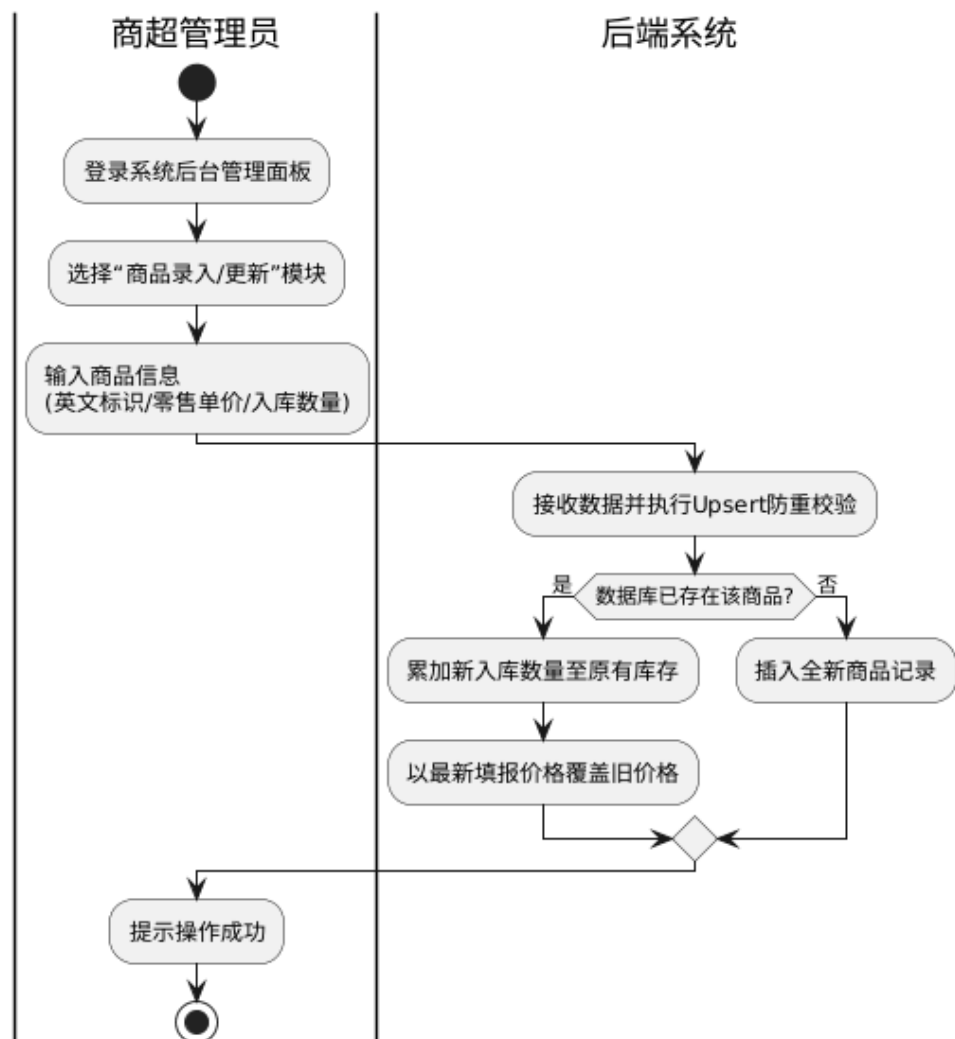


图 3-2

3.2 功能用例分析

视觉识别与定位分析 (UC-01)

- 用例名称：视觉识别与物品定位
- 用例编号：UC-01
- 参与者：顾客
- 前置条件：系统后台服务已正常启动；YOLO 模型权重文件已正确加载；操作系统已授予 Python 进程访问本地摄像头的硬件权限。
- 主要事件流 (Main Flow):
 1. 顾客浏览系统主界面的货架商品列表，选择目标商品，点击“定位寻物”按钮。
 2. 前端界面锁定该按钮防止重复提交，并通过异步请求 (Fetch API) 通知后端视觉服务。
 3. 后端视觉处理中心利用 subprocess 模块拉起独立的 YOLO 推理脚本。
 4. 视觉脚本捕捉摄像头实时帧序列，进行图像预处理，喂入 YOLO 模型进行毫秒级目标检测运算。
 5. 模型输出包含分类标签、置信度以及 Bounding Box 坐标 (x1, y1, x2, y2) 的张量数据。
 6. 系统过滤非目标物品数据，计算目标物品屏幕中心坐标，并在独立的监控窗口上绘制绿色高亮边框与红色中心准星。
 7. 顾客根据屏幕指引在物理货架上准确找到商品。
- 备选事件流 (Alternative Flow):
 - 物品未找到 (A1): 如果在视频流中持续 N 帧未检测到目标类别的置信度 $\geq$ 预设阈值 (如 0.5)，则系统在监控窗口上方显示醒目的红色文本提示“未在画面中找到物品”。
- 后置条件：顾客完成选购，按 'q' 键释放摄像头资源，系统非阻塞地安全销毁视觉推理子进程。

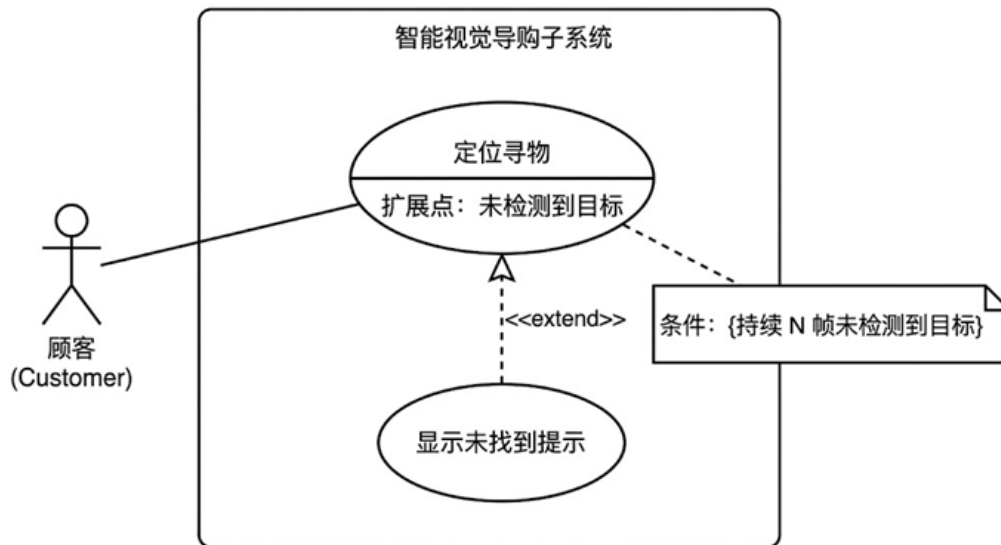


图 3-3

购物车管理与批量结算 (UC-03, UC-04)

- 用例名称：购物车管理与批量结算
- 用例编号：UC-03/04（复合用例）
- 参与者：顾客
- 前置条件：顾客身份已通过登录校验；数据库处于非锁定状态。
- 触发条件：顾客完成商品挑选后，主动点击“结算”按钮。
- 业务规则：同一商品重复加入购物车时仅累加数量；结算金额统一保留两位小数；结算接口需校验一次性请求标识（Token）以避免重复扣款。
- 主要事件流（Main Flow）：
 1. 顾客在系统前端货架列表中点击商品旁的“+ 加入购物车”按钮。
 2. 前端动态更新内存中的购物车对象，实时计算并显示当前金额总计。
 3. 顾客可在购物车面板中增减商品数量、删除单项商品，并实时查看金额变化。
 4. 顾客点击“一键刷脸结算”按钮。
 5. 前端将完整的 JSON 清单请求发送至后端结算 API。
 6. 后端开启数据库事务。遍历清单，锁定 products 表对应行记录，执行原子扣减操作： $stock = stock - buy_quantity$ 。
 7. 计算所有单品的花费金额总计。在 orders 表中为该顾客写入一条全新的销售订单记录。
 8. 后端返回订单摘要（订单 ID、金额、时间戳、购买清单），前端进行成功态渲染。
 9. 提交数据库事务。前端清空本地购物车缓存，向顾客展示模拟扣款成功的回执。

界面（包含订单 ID、消费金额）。

- 异常事件流（Exception Flow）：
 - 库存不足（E1）：在遍历结算清单时，系统检测到某单品的现有库存量小于其购买量。后端拦截交易，回滚数据库事务，并向前端返回报错信息：“【商品 A】库存不足，结算失败！”。
 - 重复提交（E2）：当顾客连续点击结算按钮导致同一 Token 重复到达时，后端仅受理首个请求，其余请求返回“订单已处理”提示，不再进行二次扣减。
 - 订单写入失败（E3）：若订单写入阶段发生数据库异常，系统执行事务回滚，并提示“系统繁忙，请稍后重试”。
- 后置条件：订单状态被持久化存储；前端恢复初始货架状态；系统静默触发库存预警用例（UC-07）。

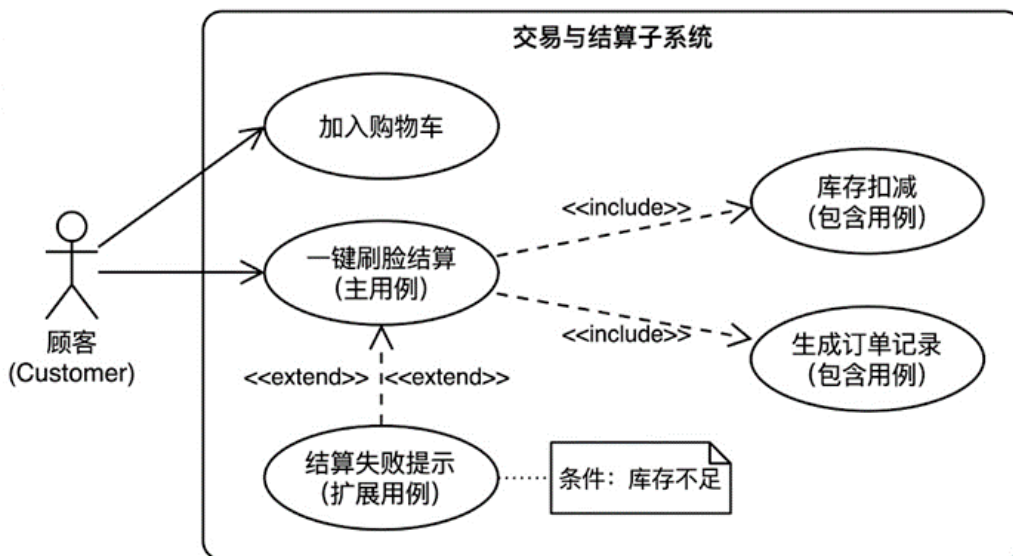


图 3-4

商品信息维护管理（UC-05）

- 用例名称：商品信息维护管理
- 用例编号：UC-05
- 参与者：商超管理员
- 前置条件：管理员已通过鉴权登录系统后台。
- 触发条件：管理员进入“商品维护”页面并提交新增或更新请求。
- 业务规则：商品英文标识必须与模型类别名一致；单价必须大于 0；库存数量必须为非负整数。
- 主要事件流（Main Flow）：

1. 管理员进入后台维护面板，输入商品的英文标识名称（需匹配 YOLO 推理脚本分类标签）。
 2. 管理员选择“新品上架”或“库存补录”模式，输入单价和入库数量（例如 50）。
 3. 管理员点击提交。后端拦截该数据。
 4. 后端执行字段合法性校验（名称、价格、库存）。
 5. 系统执行 Upsert 逻辑：先在数据库中检索是否存在同名商品。
 6. 如果存在，则更新价格并将新入库量累加至旧库存（ $\text{stock} = \text{existing_stock} + 50$ ）。
 7. 如果不存在，则创建全新商品记录（INSERT），设置初始库存。
 8. 系统记录本次操作日志（操作者、时间、变更前后值），用于审计追踪。
- 异常事件流（Exception Flow）：
 - 标签不匹配（E1）：商品名称与模型类别集不一致时，系统拒绝写入并提示“商品标签未在视觉模型中注册”。
 - 字段非法（E2）：当价格小于等于 0 或库存为负值时，系统返回参数校验失败信息。
 - 数据库不可用（E3）：写入过程中数据库连接异常，系统返回失败信息并保留原数据。
 - 后置条件：管理员接收前端“数据维护成功”的提示信息；本地数据库数据已同步更新。

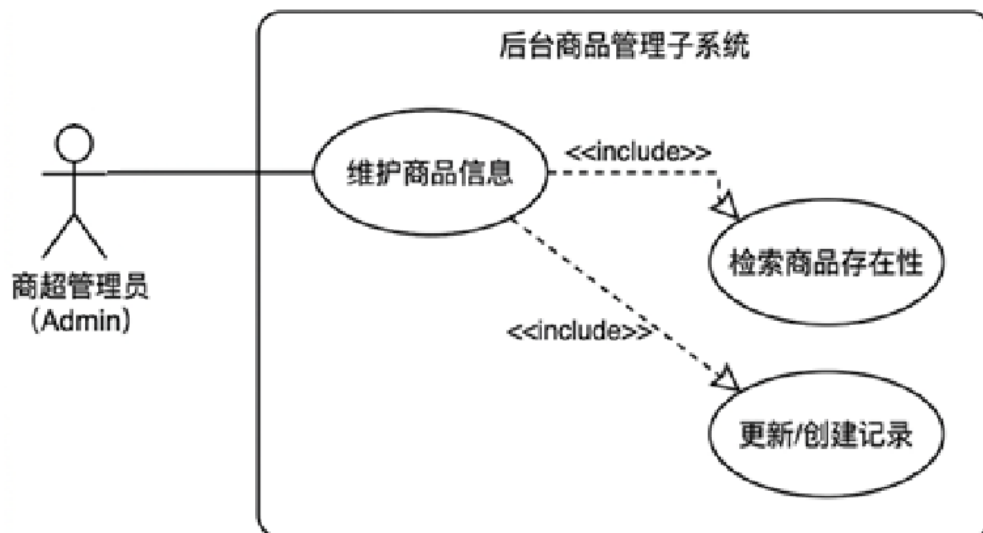


图 3-5

销售数据统计大盘（UC-06）

- 用例名称：销售数据统计大盘

- 用例编号：UC-06
- 参与者：商超管理员
- 前置条件：系统处于运营状态，数据库中存在历史订单记录。
- 触发条件：管理员打开“统计报表”页面或主动点击“刷新报表”。
- 业务规则：统计周期默认按自然日聚合；金额统一以人民币元为单位保留两位小数；Top 榜单按销量降序展示。
- 主要事件流（Main Flow）：
 1. 管理员进入后台数据报表面板。
 2. 前端自动向后端 API 请求聚合统计数据。
 3. 后端利用 SQL 聚合函数在 orders 表上执行 Group By 操作。计算核心指标：当天总营业额、总单量、总销售件数。计算销售件数 Top 5 畅销商品排行榜。
 4. 系统将计算结果封装为标准 JSON 格式并返回。
 5. 前端利用图表库（如简单的 html/js 表格）将数据进行可视化渲染呈现给管理员。
 6. 管理员可选择导出当天统计快照（CSV）用于离线分析。
- 异常事件流（Exception Flow）：
 - 无订单数据（E1）：若当前统计周期内无有效订单，系统返回零值统计并提示“暂无销售数据”。
 - 查询超时（E2）：聚合查询超时则返回缓存数据，并提示“当前展示为最近一次统计快照”。
 - 可视化渲染失败（E3）：前端图表渲染异常时自动降级为表格文本展示。
- 后置条件：统计结果被写入缓存层，供后续页面快速读取。

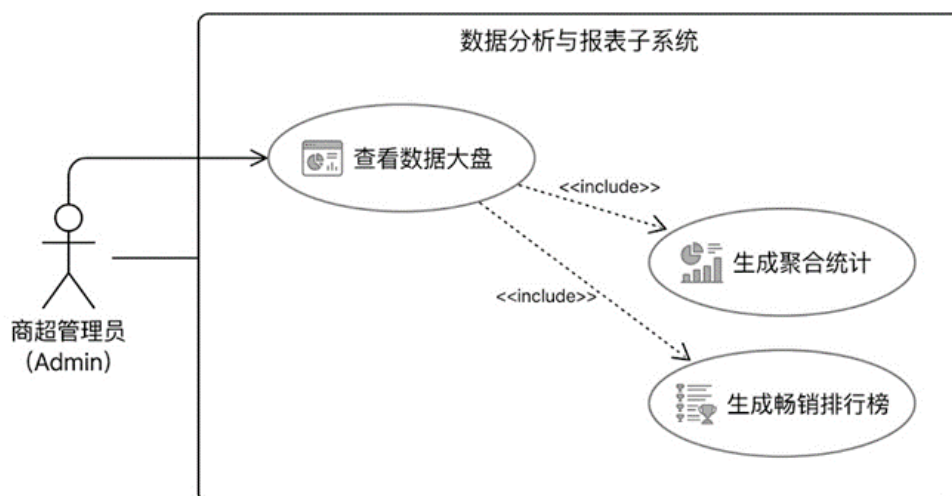


图 3-6

阈值预警与自动补货 (UC-07)

- 用例名称：阈值预警与自动补货
- 用例编号：UC-07
- 参与者：系统后台（守护进程）、模拟供应商
- 前置条件：结算事务（UC-04）成功执行且提交数据库。
- 触发条件：订单完成后库存发生变化，触发库存巡检任务。
- 业务规则：预警阈值支持按商品维度独立配置；同一商品在同一巡检周期内仅触发一次补货请求；补货默认批量为 50 件。
- 主要事件流（Main Flow）：
 1. 系统结算逻辑完成后，触发库存核查钩子（Hook）。
 2. 系统查询刚才被扣减库存的商品（例如 A）在 products 表中最新的剩余库存量。
 3. 系统判断：剩余库存 $\leq$ 预警阈值（默认 =3）？
 4. 如果是，系统后台自动组装标准化补货工单（JSON），包含：请求的外部供应商 API 地址、商品英文名、补货量（例如固定 +50 件）。
 5. 系统将报文 POST 发送至外部模拟供应商的 API 接口。
 6. 接收到供应商返回的发货确认 JSON 回执后，系统在本地数据库强制追加该商品库存（ $stock = stock + 50$ ），并生成补货系统日志供管理员查阅。
 7. 系统向管理员消息中心推送“自动补货已执行”的通知，附带商品名称与补货数量。
- 异常事件流（Exception Flow）：
 - 供应商接口不可达（E1）：若外部 API 请求失败，系统将补货任务写入重试队列并按指数退避策略重试。
 - 供应商回执异常（E2）：当回执字段缺失或状态码非法时，系统不更新库存并标记任务为“人工复核”。
 - 并发冲突（E3）：当库存记录被其他事务占用时，系统等待短时重试，超过阈值后转入补偿任务队列。
- 后置条件：供应链补给流程完成，无需人工干预。

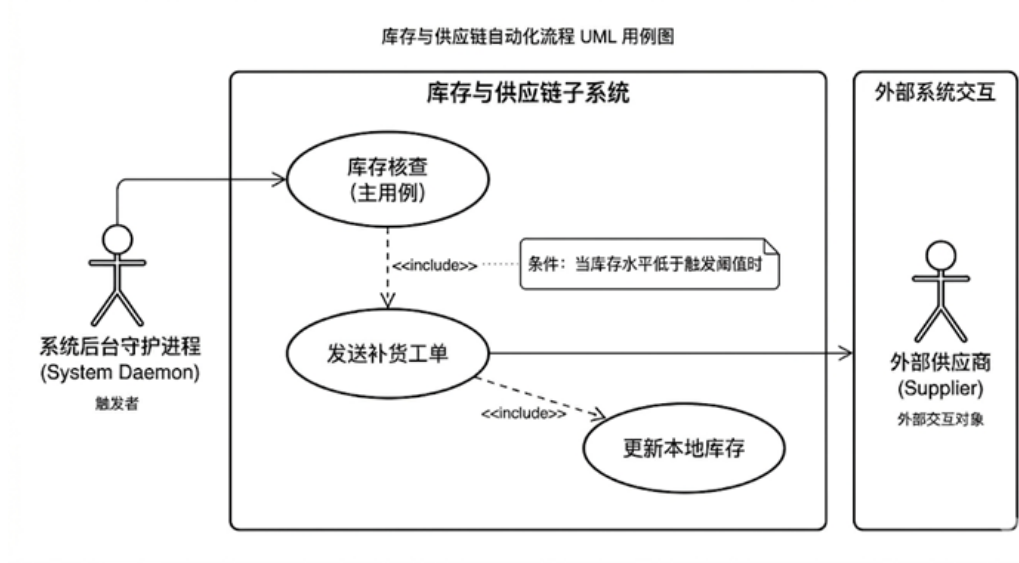


图 3-7

3.3 数据流分析

本节通过数据流图（DFD）自顶向下地分析系统内外部的数据流转过程，主要包含顶层数据流图和一层数据流图。

顶层数据流图

顶层数据流图主要界定了“智能零售系统”的边界，以及它与外部实体（顾客、管理员、供应商）之间的顶层数据交互。

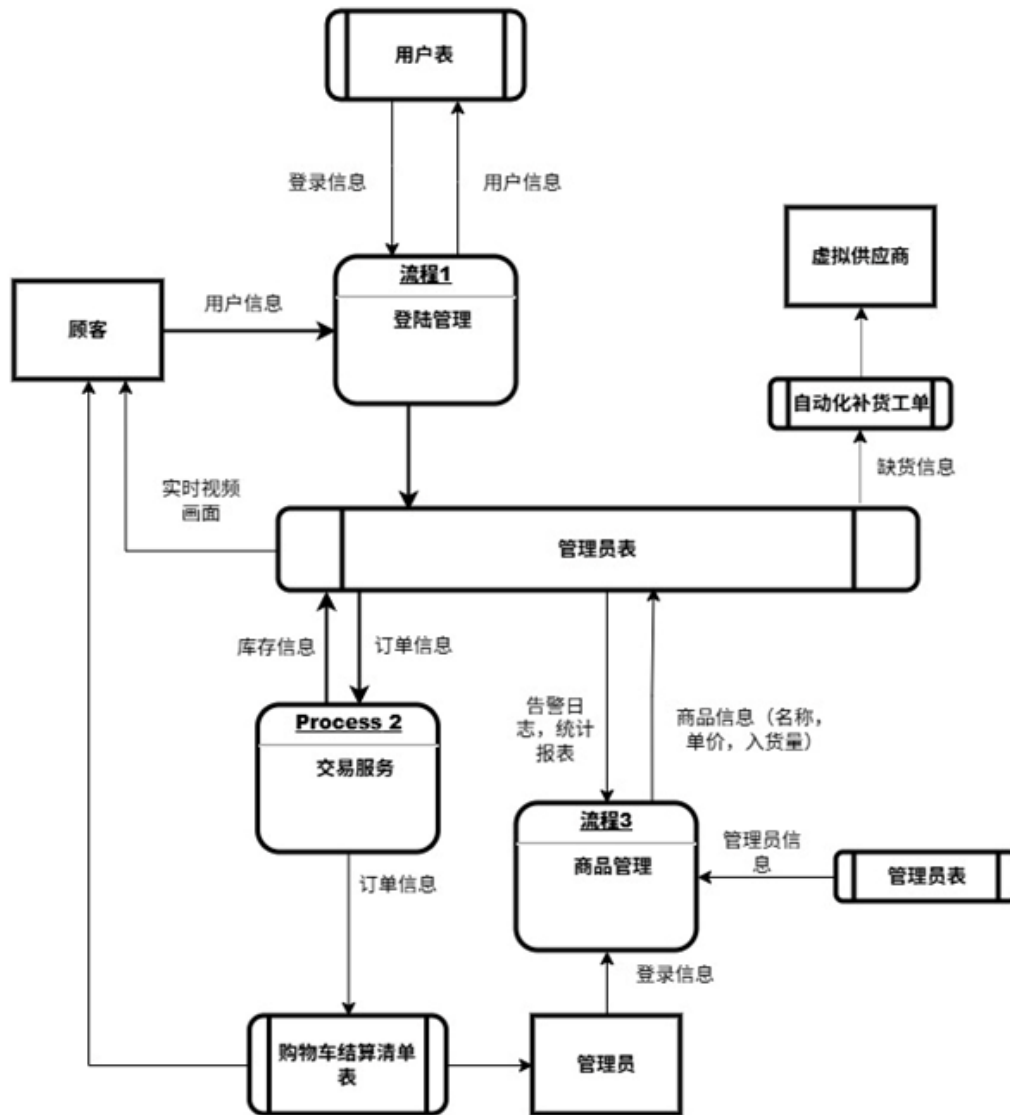


图 3-8

一层数据流图

一层数据流图主要界定了“智能零售系统”的边界，以及它与外部实体（顾客、管理员、供应商）之间的顶层数据交互。

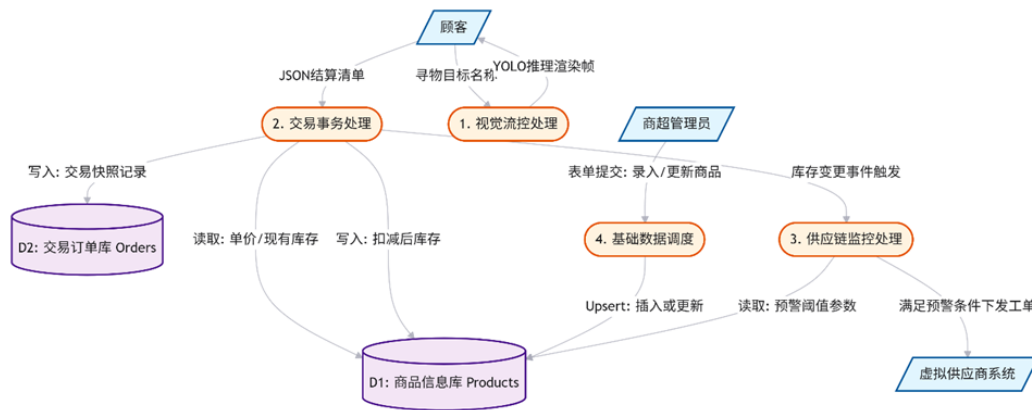


图 3-9

4 非功能性需求

4.1 性能需求

- 算法推理延迟：在配备 NVIDIA GPU 的计算环境下，YOLOv8 模型对单帧图像的推理时间应 $\leq 30\text{ms}$ ，保证视频流视觉无明显卡顿（ $>25\text{FPS}$ ）。
- 接口响应时间：除视觉处理外的所有在线数据 API（如获取商品列表、批量结算），在本地回环网络下的响应时间必须 $\leq 200\text{ms}$ 。
- 识别准确率：在光照充足、商品无严重遮挡的理想商超环境下，算法对已训练目标的识别召回率（Recall）应达到 90% 以上。

4.2 安全性需求

- 并发隔离：视觉推理模块必须与后端 Web 服务器（Uvicorn）在不同进程中运行（通过子进程拉起），防止因摄像头异常或显存溢出导致整个收银系统崩溃。
- 数据一致性：结算扣款与库存扣减必须置于同一个数据库事务（Transaction）中处理。若库存不足，必须全部回滚，严禁出现“钱已扣但库存未减”的脏写现象。
- 防注入攻击：前后端数据交互严格遵守 RESTful 规范，利用 Pydantic 模型进行强制数据类型校验，防止恶意的 SQL 注入或格式越界。

4.3 易用性需求

- 极简部署：系统需提供 requirements.txt 或 environment.yml，确保在一台全新计算机上可通过简单的命令实现一键环境复原。
- UI 容错机制：前端界面对所有网络请求异常（如后端服务未启动、接口 404）进

行捕获，并以弹窗或红色醒目文字的形式给出人类可读的提示（如“网络错误，无法连接后端”），禁止出现控制台级报错直接暴露给普通用户。

第 2 章 系统概要设计报告

1 引言

1.1 编写目的

本文档为“零售物品智能识别系统”的概要设计报告，旨在将《软件需求规格说明书》中确定的功能性与非功能性需求转化为可实施的系统体系结构、模块划分、接口契约与数据库方案。文档将作为后续详细设计、编码实现和系统测试的核心依据，确保各子系统在架构层面达成一致。

1.2 项目背景

本系统面向中小型零售商超场景，利用 YOLOv8 目标检测算法与计算机视觉技术，结合 Web 全栈开发能力，构建一套集“视觉导购、自助结算、智能库存管理”于一体的闭环平台。系统由四个子系统组成：Java 后端服务（retail-server）、Vue3 管理端（retail-admin-pro）、微信小程序客户端（retail-customer）以及 Python AI 服务（retail-ai），共同支撑商品管理、购物车结算、库存预警、摄像头巡检和智能搜索等核心业务。

1.3 相关术语

- **Spring Boot:** 基于 Java 的企业级应用开发框架，本系统采用 3.3 版本，运行于 Java 21 平台。
- **MyBatis-Plus:** MyBatis 的增强工具，简化数据持久层的 CRUD 操作。
- **JWT (JSON Web Token):** 一种无状态的用户鉴权令牌机制，本系统用于管理端和小程序端的登录凭证。
- **Vue3 + Vite:** 前端渐进式框架与构建工具，管理端基于 TypeScript 和 Element Plus 组件库。
- **微信小程序:** 运行于微信客户端的轻应用，本系统的 C 端入口。
- **FastAPI:** Python 高性能异步 Web 框架，本系统 AI 服务的运行载体。
- **OpenCV:** 开源计算机视觉库，用于摄像头采集与图像编解码。
- **YOLOv8:** 新一代实时目标检测算法，用于货架商品识别。
- **Redis:** 内存键值数据库，本系统用于推荐流缓存。
- **RESTful API:** 基于 HTTP 方法的资源型接口风格，系统内外部通信均采用此规

范。

- DTO / VO: 数据传输对象 / 视图对象, 分别用于接收请求参数和封装响应数据。
- MJPEG: Motion JPEG 视频流协议, 用于摄像头实时画面传输。
- BFF (Backend For Frontend): 为前端量身定制的后端服务层模式。

1.4 参考文献

1. 《软件工程: 实践者的研究方法》(第 9 版), Roger S. Pressman.
2. Spring Boot 3.3 官方文档 <https://spring.io/projects/spring-boot>
3. Vue3 官方文档 <https://vuejs.org/>
4. 微信小程序开发指南 <https://developers.weixin.qq.com/miniprogram/dev/>
5. FastAPI 官方文档 <https://fastapi.tiangolo.com/>
6. YOLOv8 (Ultralytics) 官方文档 <https://docs.ultralytics.com/>
7. MyBatis-Plus 官方文档 <https://baomidou.com/>

2 系统体系结构设计

2.1 系统特点分析

1. **四端分离架构:** 系统由 Java 后端 (retail-server)、Vue3 管理前端 (retail-admin-pro)、微信小程序 (retail-customer) 和 Python AI 服务 (retail-ai) 四个独立子系统组成, 各自拥有独立的代码仓库、构建工具和运行环境, 通过 RESTful HTTP/JSON 协议进行通信。
2. **统一 API 网关:** retail-server 同时承担 BFF 角色, 为管理端提供 /api/* 接口, 为小程序提供 /api/applet/* 接口, 并通过 JWT 拦截器统一鉴权。
3. **AI 能力独立部署:** retail-ai 作为独立微服务运行在 8000 端口, 仅暴露视觉识别与文本检索能力, retail-server 通过 RestTemplate 进行 HTTP 中转调用, 二者可独立扩缩容。
4. **Redis 热数据缓存:** 推荐流等高频读取数据通过 Redis 缓存, 设置 TTL 自动过期, 降低数据库查询压力。
5. **定时巡检机制:** 后端内置基于 ThreadPoolTaskScheduler 的摄像头定时巡检调度器, 可动态调整间隔和批次大小, 支持手动触发全量扫描。

2.2 系统体系结构设计

系统体系结构模式

本系统整体采用**分层架构 + 微服务**的混合体系结构模式：

- **分层架构**：retail-server 内部严格遵循 Controller → Service → Mapper 三层分层，Controller 负责请求路由与参数校验，Service 封装业务逻辑，Mapper 通过 MyBatis-Plus 完成数据持久化。
- **微服务拆分**：AI 视觉能力被独立为 retail-ai 微服务，与主后端通过 HTTP 协议松耦合集成，故障隔离性好。
- **前端 MVC/MVVM**：管理端采用 Vue3 的组合式 API + Pinia 状态管理，小程序采用微信原生框架的页面/组件模型。
- **BFF 模式**：retail-server 根据不同前端（管理端 / 小程序）的需求，提供差异化的接口路径和返回结构。

系统体系结构设计

(1) 逻辑视图设计

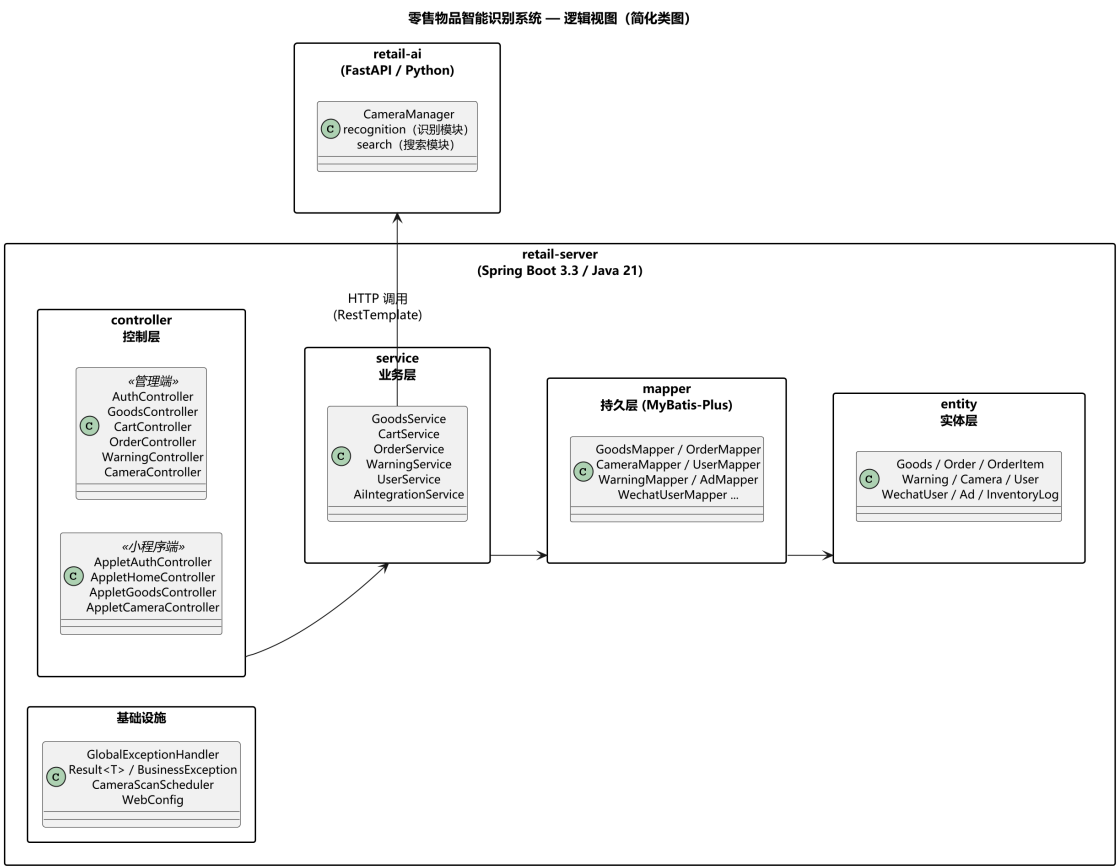


图 2-1 系统逻辑视图——简化类图

retail-server 后端代码按照职责划分为以下核心包：

- **controller 包**: 面向管理端的控制器包括 AuthController (登录鉴权)、GoodsController (商品 CRUD)、CartController (购物车)、OrderController (订单结算)、WarningController (库存告警)、CameraController (摄像头管理与巡检调度)、AdminWarningController (管理端告警列表)、FileController (文件上传); 面向小程序的控制器以 applet 子包组织, 包括 AppletAuthController、AppletHomeController、AppletGoodsController、AppletOrderController, 以及 AppletCameraController (扫码识别)。
- **service 包**: 业务逻辑层包括 GoodsService、CartService、OrderService、WarningService、UserService、OrderDetailsService、AiIntegrationService (封装对 Python AI 服务的调用) 及 AsyncCartService (异步购物车操作)。每个接口均有对应的 Impl 实现类。
- **mapper 包**: 数据持久层包含 GoodsMapper、OrderMapper、OrderItemMapper、OrderDetailMapper、CameraMapper、UserMapper、WechatUserMapper、WarningMapper、InventoryLogMapper、AdMapper、GoodsCategoryMapper 共 11 个 MyBatis-Plus Mapper 接口, 均继承 BaseMapper 泛型类。
- **entity 包**: 数据实体类与数据库表一一映射, 包括 Goods、Order、OrderItem、Warning、Camera、User、WechatUser、Ad、GoodsCategory、InventoryLog 等。
- **dto / vo 包**: DTO 用于接收前端请求 (LoginRequest、CartDTO、AppletScanRequest、CameraBindRequest 等), VO 用于封装响应数据 (AppletGoodsVO、CartVO、OrderVO、AppletAdVO 等)。
- **config 包**: 全局配置类包括 WebConfig (JWT 拦截器注册与静态资源映射)、MybatisPlusConfig (分页插件)、RestTemplateConfig (HTTP 客户端)、TaskSchedulerConfig (线程池调度器)、CorsConfig (跨域配置)。
- **exception 包**: 统一异常处理, BusinessException 为自定义业务异常, GlobalExceptionHandler 通过 @RestControllerAdvice 捕获所有异常并返回统一 JSON 结构。
- **scheduler 包**: CameraScanScheduler 实现摄像头定时巡检, 支持动态配置间隔和批次大小、手动触发、生命周期管理。
- **common 包**: Result 类封装统一的接口返回结构 (code + message + data)。
- **context 包**: UserContext 基于 ThreadLocal 存储当前登录用户 ID, 由 JwtAuthInterceptor 在请求进入时写入。

retail-ai 服务按功能划分为三个模块:

- **camera 模块:** CameraManager 类实现线程安全的摄像头管理（引用计数 + 空闲超时回收），camera routes 提供探测可用摄像头、单帧获取、MJPEG 视频流、资源释放等端点。
- **recognition 模块:** detector 模块封装 YOLO 目标检测逻辑（演示环境为模拟检测器），recognition routes 提供中心点识别和批量货架识别两个端点。
- **search 模块:** search routes 提供基于 Jaccard 相似度的文本关键词检索端点。

(2) 开发及运行视图设计

零售物品智能识别系统 — 开发运行视图（组件交互图）

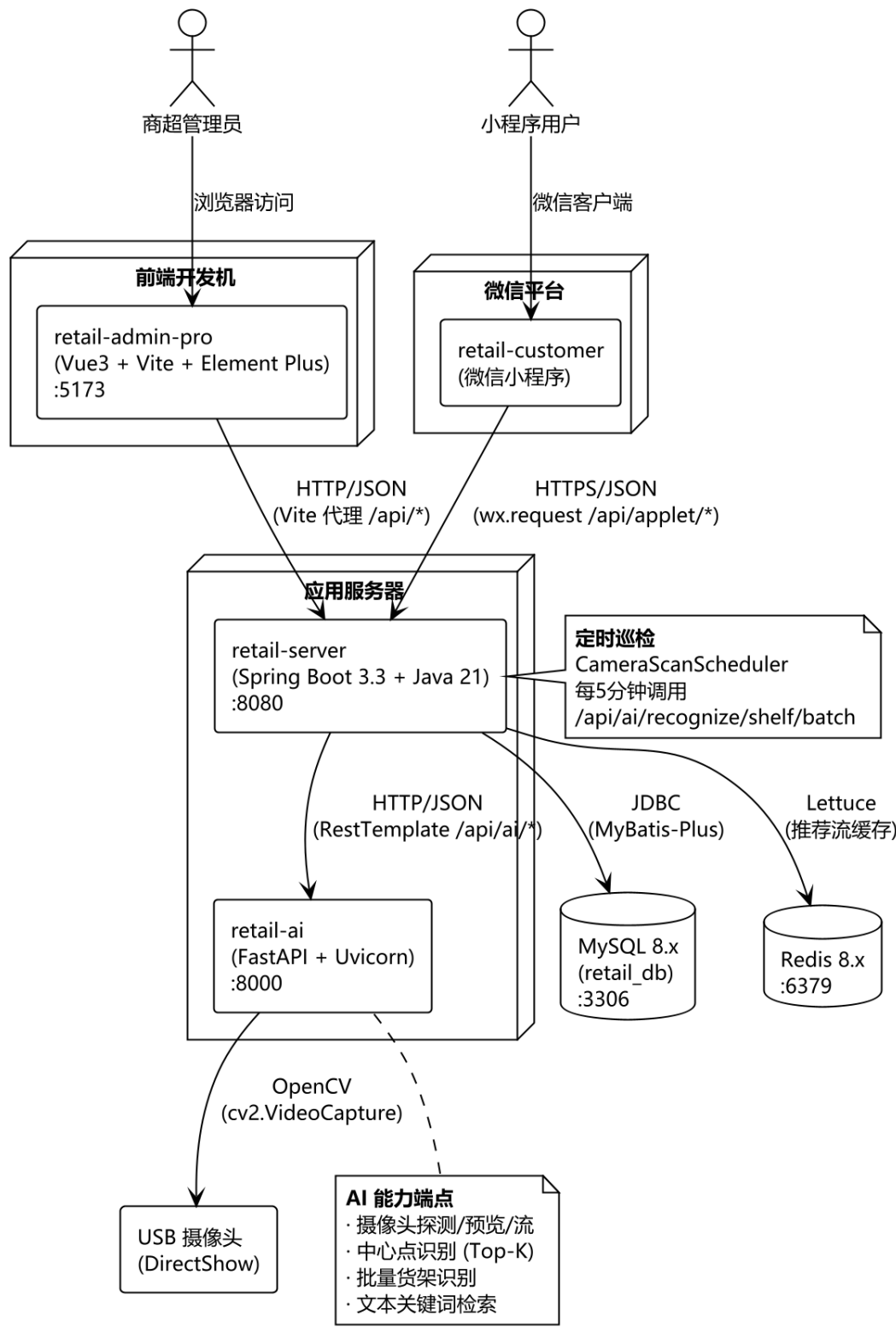


图 2-2 系统开发运行视图——组件交互图

系统开发与运行视图从子系统交互维度展示了四端之间的协作关系：

- **retail-admin-pro** (Vue3 + Vite, 开发端口 5173): 通过 Vite 反向代理将 /api 请求转发至 retail-server:8080。构建工具为 pnpm + Vite, 使用 TypeScript 编写, 集成 Element Plus 组件库、Axios HTTP 客户端、Pinia 状态管理和 Vue Router 路由。
- **retail-customer** (微信小程序): 运行于微信客户端, 通过 `wx.request` 调用 retail-server 的 /api/applet/* 接口。使用微信原生框架, 页面包括首页 (index)、搜索 (search)、购物车 (cart)、扫码 (scan)、用户中心 (user)、订单 (order)、商品详情 (goods)、充值 (recharge)、支付 (payment) 等。
- **retail-server** (Spring Boot 3.3 + Java 21, 端口 8080): 通过 Maven 构建, 依赖 spring-boot-starter-web、mybatis-plus-spring-boot3-starter、spring-boot-starter-data-redis、jjwt 等。启动后监听 8080 端口, 接受管理端和小程序的 REST 请求。需要 AI 能力时, 通过 RestTemplate 以同步 HTTP 调用 retail-ai。内置 CameraScanScheduler 定时 (默认每 5 分钟) 批量调用 retail-ai 的货架识别端点进行巡检。
- **retail-ai** (FastAPI + Uvicorn, 端口 8000): 通过 pip 安装依赖 (fastapi、uvicorn、opencv-python、numpy、pydantic), 启动后监听 8000 端口, 仅接受来自 retail-server 的内部调用。使用 OpenCV 进行摄像头操作, 通过线程安全的 CameraManager 管理摄像头生命周期。

运行时数据流如下:

1. 用户在管理端或小程序发起请求 → 到达 retail-server 的 Controller 层。
2. Controller 调用 Service 层执行业务逻辑, Service 层通过 Mapper 层访问 MySQL 数据库。
3. 若业务涉及 AI 能力 (文本搜索、图像识别、摄像头预览), Service 或 Controller 通过 RestTemplate 向 retail-ai 发起 HTTP 请求。
4. retail-ai 执行视觉推理或文本检索后返回 JSON 结果。
5. retail-server 整合数据库数据与 AI 结果, 封装为统一 Result 结构返回前端。
6. 推荐流等热点数据写入 Redis 缓存, 后续请求直接从缓存读取。

(3) 部署视图设计

零售物品智能识别系统 — 部署视图 (部署图)

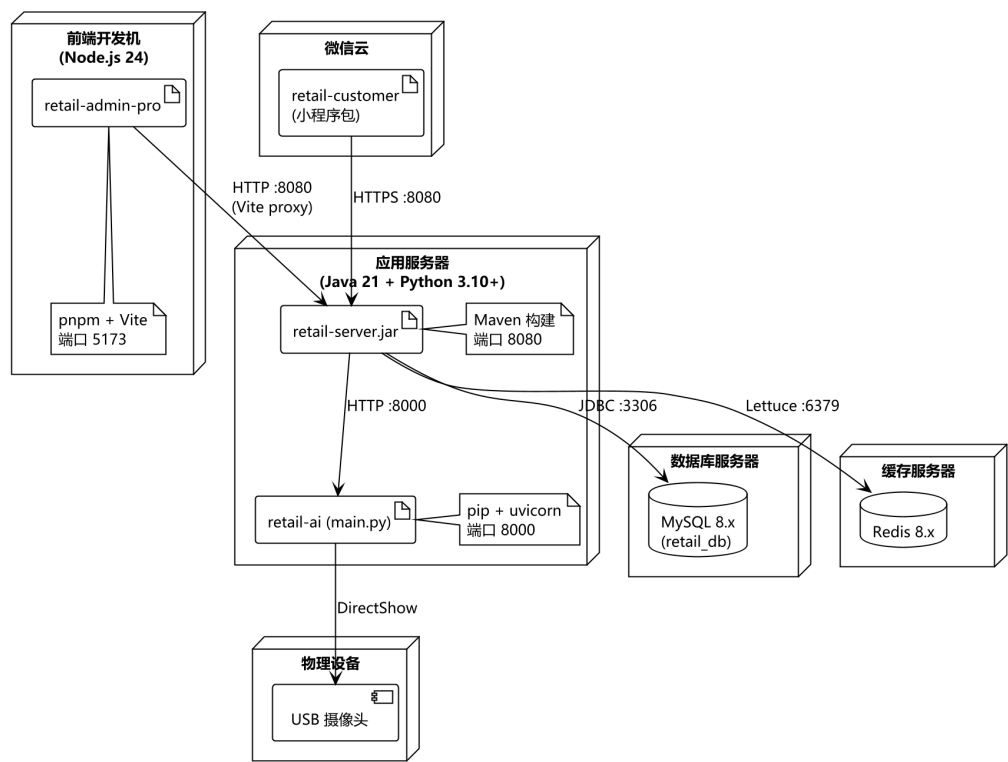


图 2-3 系统部署视图——部署图

系统各组件部署节点说明如下：

部署节点	运行内容	端口	运行环境
应用服务器	retail-server（Spring Boot）	8080	Java 21 + Maven
应用服务器	retail-ai（FastAPI + Uvicorn）	8000	Python 3.10+
前端开发机	retail-admin-pro（Vite Dev Server）	5173	Node.js 24
微信平台	retail-customer（小程序包）	—	微信客户端
数据库服务器	MySQL 8.x（retail_db 数据库）	3306	—
缓存服务器	Redis 8.x	6379	—
物理设备	USB 摄像头	—	DirectShow

表 2-1 系统部署节点说明

部署拓扑说明：

- 开发阶段 retail-server、retail-ai、MySQL、Redis 均可部署在同一台开发机上，管理端通过 Vite 反向代理避免跨域。

- 生产环境中 retail-server 与 retail-ai 建议分离部署，retail-ai 部署在配备 GPU 或摄像头的专用节点。
- 小程序端通过公网 HTTPS 访问 retail-server（需配置域名与 SSL 证书），管理端可通过 Nginx 部署静态资源。
- 摄像头设备通过 USB 连接到 retail-ai 所在节点，由 OpenCV 的 DirectShow 后端驱动采集。

3 接口设计

3.1 外部接口

系统对外暴露的 RESTful API 统一采用 JSON 格式，响应结构为 {code, message, data}。以下按功能模块分组列出所有外部接口。

认证模块

方法	路径	说明
POST	/api/auth/login	管理员登录，返回 JWT Token
POST	/api/applet/auth/login	小程序登录（微信 code 换 Token）

表 3-1 认证模块接口

文件管理

方法	路径	说明
POST	/api/upload	上传文件，返回可访问 URL

表 3-2 文件管理接口

商品管理（管理端）

方法	路径	说明
GET	/api/goods/page	分页查询商品，支持名称模糊搜索
POST	/api/goods	新增商品
PUT	/api/goods	修改商品（含库存补货日志记录）
DELETE	/api/goods/{id}	删除商品

表 3-3 商品管理（管理端）接口

购物车模块

方法	路径	说明
POST	/api/cart/add	加入或修改购物车商品数量
GET	/api/cart/list	查询当前用户购物车列表
DELETE	/api/cart/clear	清空当前用户购物车

表 3-4 购物车模块接口

订单模块

方法	路径	说明
POST	/api/order/checkout	购物车一键结算，生成订单
GET	/api/order/list	查询当前用户历史订单

表 3-5 订单模块接口

库存告警模块

方法	路径	说明
GET	/api/warning/page	分页查询告警列表，按状态筛选
PUT	/api/warning/{id}/resolve	将告警标记为已处理
GET	/api/admin/warning/list	管理端未处理告警列表

表 3-6 库存告警模块接口

摄像头管理模块（管理端）

方法	路径	说明
GET	/api/admin/camera/list	查询已绑定摄像头列表
GET	/api/admin/camera/available_hardware	获取可用物理摄像头索引
POST	/api/admin/camera	绑定新增摄像头
PUT	/api/admin/camera	编辑摄像头货架绑定
DELETE	/api/admin/camera/{id}	删除摄像头
GET	/api/admin/camera/preview/{no}	预览摄像头单帧画面
GET	/api/admin/camera/stream/{no}	MJPEG 实时视频流代理
POST	/api/admin/camera/stream/{no}/stop	释放摄像头视频流资源

表 3-7 摄像头管理模块（管理端）接口

巡检调度模块（管理端）

方法	路径	说明
GET	/api/admin/camera/scheduler/config	查询巡检调度配置
PUT	/api/admin/camera/scheduler/config	更新巡检间隔与批次
POST	/api/admin/camera/scheduler/trigger	手动触发全量巡检

表 3-8 巡检调度模块（管理端）接口

小程序首页模块

方法	路径	说明
GET	/api/applet/home/ads	轮播广告列表（免鉴权）
GET	/api/applet/home/recommend	推荐流（Redis 缓存分页）

表 3-9 小程序首页模块接口

小程序搜索识别模块

方法	路径	说明
GET	/api/applet/search/text	文本关键词搜索商品
POST	/api/applet/search/image	图像搜索商品
POST	/api/applet/scan	扫码识别（返回商品详情）
POST	/api/applet/goods/listByIds	按 ID 批量查询商品详情

表 3-10 小程序搜索识别模块接口

所有需要鉴权的接口（除标注“免鉴权”外）需在请求头携带 `Authorization: Bearer {token}`，由 `JwtAuthInterceptor` 统一校验。统一响应结构 `Result` 定义为：

字段	类型	说明
code	Integer	业务状态码（200 成功，4xx 客户端错误，5xx 服务端错误）
message	String	操作描述信息
data	T（泛型）	实际返回数据

表 3-11 统一响应结构 `Result`

3.2 内部接口

`retail-server` 通过 `RestTemplate` 以同步 HTTP 调用 `retail-ai` 的内部 API。这些接口不直接暴露给前端，仅用于后端服务间通信。

方法	路径	说明
GET	/api/ai/cameras/available	探测本机可用物理摄像头索引
GET	/api/ai/cameras/frame/{index}	获取指定摄像头单帧（base64）
GET	/api/ai/cameras/stream/{index}	MJPEG 实时视频流
POST	/api/ai/cameras/stream/{index}/stop	主动释放摄像头资源
POST	/api/ai/recognize/center	中心点识别 Top-K（扫码用）
POST	/api/ai/recognize/shelf/batch	批量货架识别（巡检用）
POST	/api/ai/search/text	文本关键词检索商品 ID

表 3-12 接口接口

调用约定：

- 基础地址通过 application.yml 中的 ai.service-url 配置，默认值为 http://localhost:8000。
- 请求与响应均为 JSON 格式，Python 端使用 Pydantic 模型进行参数校验。
- retail-server 对 RestClientException 进行统一捕获，转换为 502 业务异常返回前端。
- 连接超时 3 秒，读取超时 10 秒（在 RestTemplateConfig 中配置）。

4 系统数据库设计

4.1 概念数据库设计

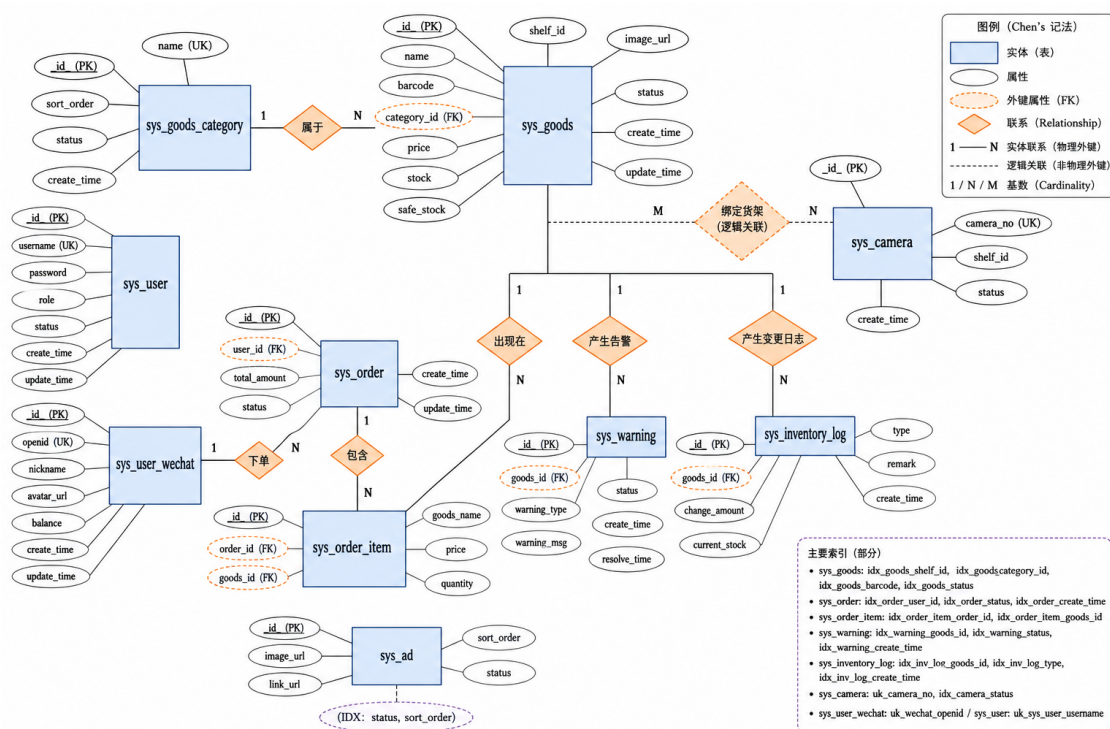


图 4-1 系统概念数据库设计——ER 图

系统数据库 retail_db 共包含 10 个实体，核心实体及联系描述如下：

- **系统用户 (sys_user) 与订单 (sys_order)：**一对多关系，一个管理端用户可创建多个订单。
- **小程序用户 (sys_user_wechat) 与订单 (sys_order)：**一对多关系，一个小程序用户可创建多个订单。
- **订单 (sys_order) 与订单明细 (sys_order_item)：**一对多关系，一个订单包含多条明细。
- **商品 (sys_goods) 与订单明细 (sys_order_item)：**一对多关系，一个商品可出现

在多条订单明细中。

- **商品分类 (sys_goods_category) 与商品 (sys_goods):** 一对多关系，一个分类下可有多个商品。
- **商品 (sys_goods) 与库存告警 (sys_warning):** 一对多关系，一个商品可触发多条告警记录。
- **商品 (sys_goods) 与库存变更日志 (sys_inventory_log):** 一对多关系，一个商品有多条库存变更流水。
- **摄像头 (sys_camera) 与商品 (sys_goods):** 通过 shelf_id 字段逻辑关联，一个摄像头监控一个货架，一个货架上可摆放多个商品。
- **首页广告 (sys_ad):** 独立实体，与其他实体无外键关联。

4.2 逻辑数据库设计

以下按表逐一列出数据库 retail_db 的全部 10 张表的逻辑结构。

1. 系统用户表 (sys_user)

字段名	数据类型	约束	说明
id	BIGINT UNSIGNED	PK, 自增	主键 ID
username	VARCHAR(64)	NOT NULL, UK	用户名
password	VARCHAR(255)	NOT NULL	密码
role	ENUM	NOT NULL	角色 (admin/customer)
status	TINYINT	NOT NULL, 默认 1	状态 (1 启用/0 禁用)
create_time	DATETIME	NOT NULL	创建时间
update_time	DATETIME	NOT NULL	更新时间 (自动维护)

表 4-1 sys_user 表结构

2. 商品分类表 (sys_goods_category)

字段名	数据类型	约束	说明
id	BIGINT UNSIGNED	PK, 自增	分类 ID
name	VARCHAR(64)	NOT NULL, UK	分类名称
sort_order	INT	NOT NULL, 默认 0	排序值
status	TINYINT	NOT NULL, 默认 1	状态
create_time	DATETIME	NOT NULL	创建时间

表 4-2 sys_goods_category 表结构

3. 商品基础表 (sys_goods)

字段名	数据类型	约束	说明
id	BIGINT UNSIGNED	PK, 自增	主键 ID
name	VARCHAR(128)	NOT NULL	商品名称
barcode	VARCHAR(64)	可空, 索引	条形码
category_id	BIGINT UNSIGNED	可空, 索引	分类 ID
price	DECIMAL(10,2)	NOT NULL	商品价格
stock	INT	NOT NULL, 默认 0	库存数量
safe_stock	INT	NOT NULL, 默认 10	安全库存阈值
shelf_id	VARCHAR(64)	NOT NULL, 索引	货架编号
image_url	VARCHAR(255)	可空	商品图片 URL
status	TINYINT	NOT NULL, 默认 1	上/下架状态
create_time	DATETIME	NOT NULL	创建时间
update_time	DATETIME	NOT NULL	更新时间

表 4-3 sys_goods 表结构

4. 订单主表 (sys_order)

字段名	数据类型	约束	说明
id	BIGINT UNSIGNED	PK, 自增	订单 ID
user_id	BIGINT UNSIGNED	NOT NULL, 索引	用户 ID
total_amount	DECIMAL(10,2)	NOT NULL	订单总金额
status	VARCHAR(32)	NOT NULL	订单状态
create_time	DATETIME	NOT NULL, 索引	创建时间
update_time	DATETIME	NOT NULL	更新时间

表 4-4 sys_order 表结构

5. 订单明细表 (sys_order_item)

字段名	数据类型	约束	说明
id	BIGINT UNSIGNED	PK, 自增	明细 ID
order_id	BIGINT UNSIGNED	NOT NULL, 索引	订单 ID
goods_id	BIGINT UNSIGNED	NOT NULL, 索引	商品 ID
goods_name	VARCHAR(128)	NOT NULL	商品名称快照
price	DECIMAL(10,2)	NOT NULL	商品单价快照
quantity	INT	NOT NULL, 默认 1	购买数量

表 4-5 sys_order_item 表结构

6. 库存告警表 (sys_warning)

字段名	数据类型	约束	说明
id	BIGINT UNSIGNED	PK, 自增	告警 ID
goods_id	BIGINT UNSIGNED	NOT NULL, 索引	商品 ID
warning_type	VARCHAR(32)	NOT NULL	告警类型
warning_msg	VARCHAR(255)	NOT NULL	告警内容
status	TINYINT	NOT NULL, 默认 0	处理状态
create_time	DATETIME	NOT NULL, 索引	创建时间
resolve_time	DATETIME	可空	处理时间

表 4-6 sys_warning 表结构

7. 库存变更日志表 (sys_inventory_log)

字段名	数据类型	约束	说明
id	BIGINT UNSIGNED	PK, 自增	日志 ID
goods_id	BIGINT UNSIGNED	NOT NULL, 索引	商品 ID
change_amount	INT	NOT NULL	变动数量（正/负）
current_stock	INT	NOT NULL	变动后当前库存
type	VARCHAR(32)	NOT NULL, 索引	日志类型
remark	VARCHAR(255)	可空	备注
create_time	DATETIME	NOT NULL, 索引	创建时间

表 4-7 sys_inventory_log 表结构

8. 小程序用户表 (sys_user_wechat)

字段名	数据类型	约束	说明
id	BIGINT UNSIGNED	PK, 自增	主键 ID
openid	VARCHAR(128)	NOT NULL, UK	微信用户唯一标识
nickname	VARCHAR(64)	可空	昵称
avatar_url	VARCHAR(255)	可空	头像 URL
balance	DECIMAL(10,2)	NOT NULL, 默认 0	钱包余额
create_time	DATETIME	NOT NULL	创建时间
update_time	DATETIME	NOT NULL	更新时间

表 4-8 sys_user_wechat 表结构

9. 首页轮播广告表 (sys_ad)

字段名	数据类型	约束	说明
id	BIGINT UNSIGNED	PK, 自增	主键 ID
image_url	VARCHAR(255)	NOT NULL	轮播图 URL
link_url	VARCHAR(255)	可空	点击跳转链接
sort_order	INT	NOT NULL, 默认 0	排序值
status	TINYINT	NOT NULL, 默认 1	启用状态

表 4-9 sys_ad 表结构

10. 摄像头管理表 (sys_camera)

字段名	数据类型	约束	说明
id	BIGINT UNSIGNED	PK, 自增	摄像头 ID
camera_no	VARCHAR(32)	NOT NULL, UK	摄像头编号
shelf_id	VARCHAR(64)	NOT NULL	绑定货架编号
status	TINYINT	NOT NULL, 默认 1	状态（1 正常/0 停用）
create_time	DATETIME	NOT NULL	创建时间

表 4-10 sys_camera 表结构

5 系统出错处理设计

5.1 出错信息

系统通过 GlobalExceptionHandler (@RestControllerAdvice) 统一拦截所有异常，将其转换为标准 Result 结构返回，避免堆栈信息直接暴露给前端。以下为系统可能出现的出错信息一览表：

业务码	触发场景	提示信息
400	请求参数缺失或非法	“商品 ID 不能为空”“分页参数必须大于 0”等
400	请求体 JSON 格式错误	“请求体 JSON 格式错误”
400	camera_no 已被绑定	“该 camera_no 已被绑定”
400	小程序登录 code 非法	“演示环境仅支持 code=test_code”
401	未登录或 Token 过期/无效	“未登录或 Token 无效”
404	资源不存在	“商品不存在或修改失败”“摄像头不存在”“告警不存在”
404	接口路径错误	“接口不存在，请检查请求路径”
405	请求方法不匹配	“请求方法不支持”
415	Content-Type 不支持	“Content-Type 不支持，请使用 application/json”
500	数据库操作失败	“新增商品失败”“处理告警失败”等
500	摄像头预览数据为空	“摄像头预览数据为空”
500	未知系统异常	“服务器内部异常”
502	Python AI 服务调用失败	“调用 Python XX 服务失败”

表 5-1 系统出错信息一览

前端出错提示设计：

- 管理端（retail-admin-pro）：通过 Axios 拦截器统一捕获非 200 响应，调用 checkStatus 函数根据 HTTP 状态码映射为 Element Plus 的 ElMessage 弹窗提示；403/404/500 等页面级错误显示专用错误页面。
- 小程序端（retail-customer）：在 wx.request 的 fail 回调和 success 中判断 res.data.code，非 200 时通过 wx.showToast 显示友好错误提示。

5.2 补救措施

1. **JWT 令牌过期**：前端拦截 401 响应后自动清除本地 Token 并跳转登录页面，要求用户重新认证。管理端通过 Axios 拦截器实现，小程序端在请求封装层判断后

调用 `wx.redirectTo` 跳转至登录页。

2. **AI 服务宕机:** retail-server 对所有 RestTemplate 调用进行 try-catch 包裹, 捕获 Rest-ClientException 后返回 502 错误码。AI 服务不可用不影响商品管理、购物车、订单结算等核心功能的正常运行。
3. **数据库事务失败:** 订单结算 (checkout) 在 Service 层使用数据库事务, 库存扣减与订单写入在同一事务中执行。若任一步骤失败, 事务自动回滚, 保证不出现“钱已扣但库存未减”的数据不一致。
4. **摄像头资源泄露:** retail-ai 的 CameraManager 采用引用计数机制, 每次 acquire 增加计数、release 减少计数。后台 Reaper 线程每 5 秒扫描一次, 对引用计数归零且空闲超过 60 秒的摄像头自动执行硬件释放。服务关闭时 on_shutdown 钩子释放所有摄像头。
5. **Redis 缓存不可用:** 推荐流接口在 Redis 无数据时返回 204 (无更多数据), 不影响用户使用其他功能。购物车使用 Redis 存储时, 异常降级为返回空列表。
6. **并发请求冲突:** 商品库存更新通过 MyBatis-Plus 的条件更新保证数据安全; 摄像头管理器内部使用 threading.Lock 保证线程安全。
7. **文件上传失败:** FileController 校验上传文件大小和类型, 异常时返回明确错误信息, 不会导致服务崩溃。
8. **网络中断:** 前端均设置请求超时 (管理端 Axios 超时配置、小程序 wx.request timeout), 超时后显示“网络异常”提示, 用户可手动重试。

第 3 章 系统详细设计报告

1 引言

1.1 编写目的

本文档在《软件需求规格说明书》和《系统概要设计报告》的基础上，对系统核心模块进行部件级细化设计，明确前后端交互、类与接口职责、关键业务时序及异常处理策略，为编码实现、联调测试和后期运维提供统一依据。

1.2 项目背景

“零售物品智能识别系统”面向商超场景，目标是打通顾客侧“搜索/识别/下单”链路与管理侧“商品维护/库存预警/摄像头巡检”链路。当前项目由四个子系统组成：

- **retail-admin-pro**: 管理端前端（Vue3 + Vite）。
- **retail-customer**: 微信小程序客户端。
- **retail-server**: Spring Boot 后端服务，负责业务编排与数据一致性。
- **retail-ai**: FastAPI AI 服务，提供摄像头、识别、搜索能力。

1.3 参考文献

1. 《软件工程：实践者的研究方法》
2. Spring Boot 3.3 官方文档
3. FastAPI 官方文档
4. 微信小程序开发文档
5. MyBatis-Plus 官方文档

2 系统功能需求概要

系统按业务闭环划分为三个主模块：

1. **智能视觉导购模块**：支持文本检索、图片识别、货架定位。
2. **交易与结算模块**：支持购物车管理、库存校验、订单生成、支付确认。
3. **后台商品与库存监控模块**：支持商品维护、摄像头管理、告警处理、定时巡检。

模块间关系如下：

- 顾客侧通过小程序与后端交互，后端按需转发 AI 能力调用。
- 管理员侧通过管理端维护主数据与调度参数。
- 交易成功后触发库存与告警联动，低库存时由后台机制继续触发补给流程。

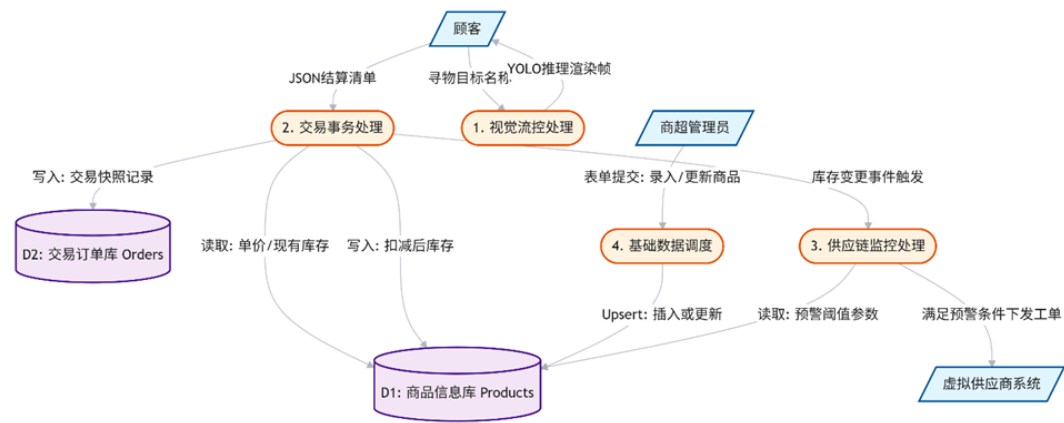


图 3-1 系统一层数据流（用于功能分解映射）

3 智能视觉导购模块详细设计

3.1 功能模块概述

智能视觉导购模块聚焦“找得到、看得见、加得进购物车”，主要功能为：

- 文本检索：小程序调用 `/api/applet/search/text`，后端中转 `/api/ai/search/text`。
- 图片识别：小程序上传 base64 图像至 `/api/applet/scan`，后端调用 `/api/ai/recognize/center` 返回 Top-K 商品候选。
- 商品详情补全：通过 `/api/applet/goods/listByIds` 将识别出的 ID 列表转换为可展示的商品信息。

3.2 用户交互界面设计

顾客端对应页面与交互如下：

1. 查询页（`pages/search`）：输入关键词，实时节流检索，展示结果与推荐流。
2. 识别页（`pages/scan`）：拍照或相册选图后，发起识别并在页面展示候选商品。
3. 商品详情页（`pages/goods`）：查看单品信息并加入购物车。

交互设计原则：

- 请求期间给出加载态，避免重复点击。
- 识别失败和空结果分别提示，保证用户可理解性。

- 商品加入购物车后即时反馈，并更新购物车角标。

3.3 类及接口细化设计

核心类与职责

- AppletCameraController: 接收 image_base64，调用 AI 聚合服务。
- AppletGoodsController: 提供文本检索、图片检索、按 ID 批量查详情。
- AiIntegrationServiceImpl: 统一封装与 Python AI 的 HTTP 调用。
- recognition/routes.py: 实现中心点识别与批量货架识别。
- search/routes.py: 实现文本检索（Jaccard 相似度 + 稳定打分）。

关键接口说明

接口	方法	说明
/api/applet/search/text	GET	文本检索，参数 keyword、k
/api/applet/scan	POST	图片识别，参数 image_base64、k
/api/applet/goods/listByIds	POST	按商品 ID 列表批量查询详情
/api/ai/recognize/center	POST	AI 中心点识别（内部接口）
/api/ai/search/text	POST	AI 文本检索（内部接口）

表 3-1 视觉导购模块关键接口

时序细化（文字时序）

识别寻物主流程的时序如下：

- 顾客在小程序选择图片并提交。
- 小程序调用 /api/applet/scan。
- 后端调用 AI 服务 /api/ai/recognize/center 获取候选 ID。
- 后端查询商品主数据并补全名称、价格、库存、货架号。
- 后端返回候选列表；前端展示并支持加入购物车。

4 交易与结算模块详细设计

4.1 功能模块概述

交易与结算模块负责购物车数据管理、订单结算、库存扣减及订单查询。关键约束为“库存扣减与订单写入必须同事务提交，失败则整体回滚”。

4.2 用户交互界面设计

顾客端交互主要在以下页面完成：

- **购物车页 (pages/cart)**: 增减数量、勾选商品、清空购物车、进入结算。
- **订单页 (pages/order)**: 确认结算、查看历史订单、按状态筛选。
- **支付页 (pages/payment)**: 余额/微信支付方式选择与支付确认。

界面状态设计：

1. 购物车为空时禁止结算。
2. 结算中使用按钮锁定和 Loading，防止重复提交。
3. 支付成功后跳转用户中心并刷新订单状态。

4.3 类及接口细化设计

核心类与职责

- **CartController + CartServiceImpl**: 维护 Redis 购物车。
- **OrderController + OrderServiceImpl**: 执行结算事务与订单查询。
- **GoodsMapper**: 提供 `selectByIdForUpdate` 与 `decreaseStock` 原子扣减能力。
- **OrderItemMapper、InventoryLogMapper**: 写入明细与库存流水。

结算流程关键逻辑

1. 从 Redis 读取用户购物车并校验合法性。
2. 对商品 ID 排序后逐条加锁 (`FOR UPDATE`) 降低并发死锁概率。
3. 校验库存并执行条件扣减 (`stock >= quantity`)。
4. 写入库存日志，低于阈值时生成库存告警。
5. 写入订单主表与订单明细表。
6. 事务提交后异步清空 Redis 购物车。

关键接口说明

接口	方法	说明
/api/cart/add	POST	加入/修改购物车商品数量
/api/cart/list	GET	查询当前用户购物车
/api/order/checkout	POST	管理端/通用结算接口
/api/applet/order/checkout	POST	小程序结算接口（可携带清单）
/api/applet/order/pay	POST	小程序订单支付确认
/api/order/list	GET	查询订单历史

表 3-2 交易与结算模块关键接口

5 后台商品与库存监控模块详细设计

5.1 功能模块概述

该模块面向管理员与系统后台任务，完成商品全生命周期管理、库存告警治理、摄像头绑定与巡检调度。核心能力包括：

- 商品分页查询、新增、编辑、删除、图片上传。
- 告警分页查询、告警处理、未处理告警聚合。
- 摄像头绑定、预览、流式代理、调度参数动态更新。
- 定时巡检调用 AI 批量识别并同步货架商品信息。

5.2 用户交互界面设计

管理端对应页面如下：

1. 商品管理页（`views/goods/manage`）：搜索、增删改、库存可视标签、图片上传。
2. 视觉调度页（`views/camera/schedule`）：摄像头绑定、预览、巡检配置、手动触发。
3. 数据看板页（`views/dashboard/dataVisualize`）：展示销售、库存和告警概览。

5.3 类及接口细化设计

核心类与职责

- GoodsController：商品 CRUD 与库存补货日志触发。
- WarningController / AdminWarningController：告警分页与待处理清单。
- CameraController：摄像头管理与流代理接口。

- CameraScanScheduler：定时任务调度、分批调用 AI、同步货架信息。
- GlobalExceptionHandler：统一异常转换与错误码输出。

调度时序细化（文字时序）

1. 定时器按 interval-minutes 触发巡检任务。
2. 读取状态正常的摄像头列表并按 batch-size 分批。
3. 后端向 /api/ai/recognize/shelf/batch 提交批量识别请求。
4. 收到 detected_goods_ids 后回写商品 shelf_id。
5. 输出巡检日志，供管理员在页面和日志系统中查看。

关键接口说明

接口	方法	说明
/api/goods/page	GET	商品分页查询
/api/goods	POST/PUT	商品新增与编辑
/api/warning/page	GET	告警分页查询
/api/warning/{id}/resolve	PUT	告警处理
/api/admin/camera/list	GET	摄像头列表
/api/admin/camera/scheduler/config	GET/PUT	巡检参数查询与更新
/api/admin/camera/scheduler/trigger	POST	手动触发巡检

表 3-3 后台监控模块关键接口

本章从模块职责、界面交互、类与接口、关键时序四个层面完成了详细设计，后续测试用例与部署方案均以本章为实现依据。

第 4 章 系统测试报告

1 项目简介

1.1 项目业务功能介绍

系统业务覆盖“顾客购前导购、购中结算、购后库存联动”全链路，核心流程如下：

- 1. 顾客通过小程序进行文本检索或图片识别，定位目标商品。
- 2. 顾客将商品加入购物车并发起结算。
- 3. 后端校验库存并生成订单，写入库存日志与告警信息。
- 4. 管理员在管理端维护商品与摄像头，并通过巡检调度监控货架状态。

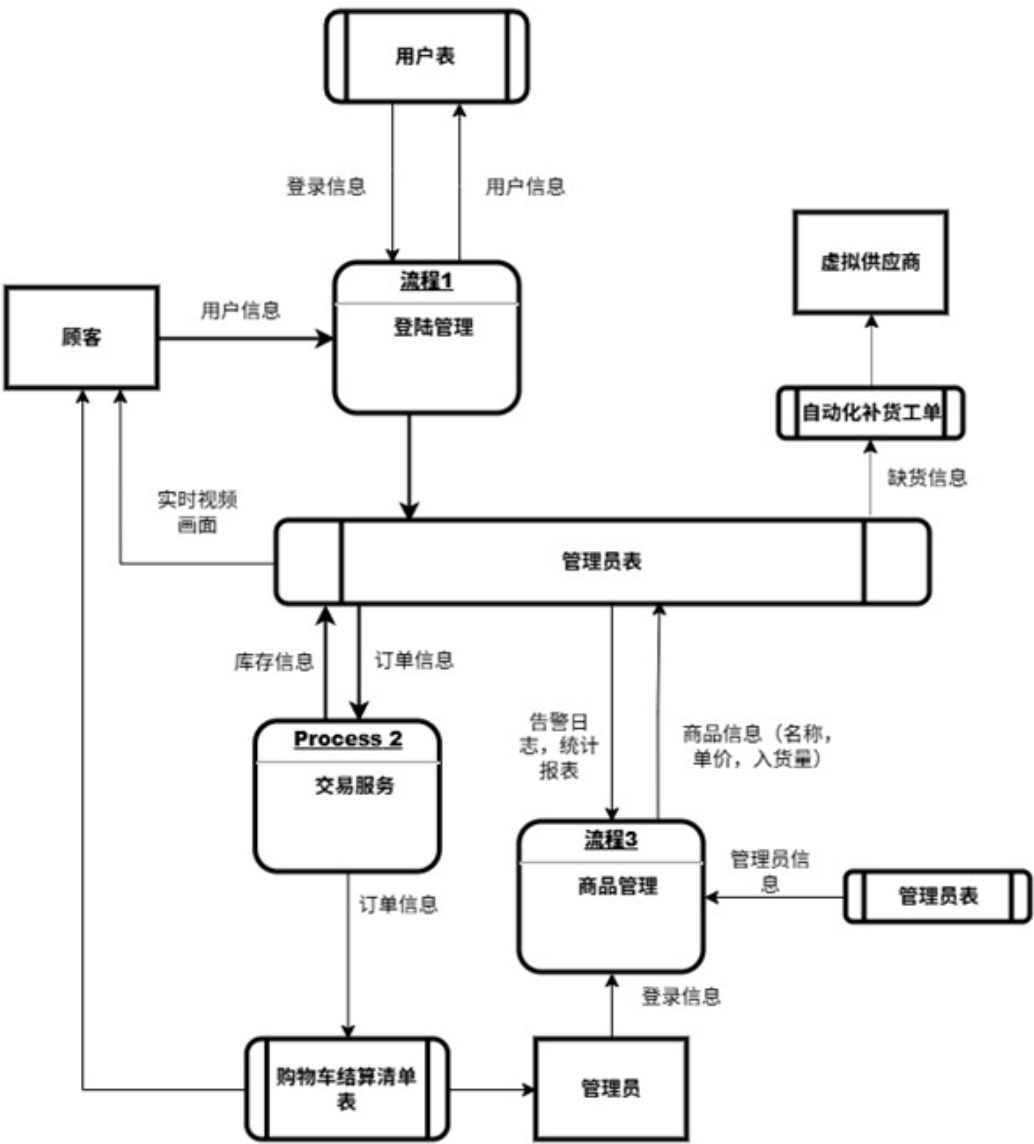


图 4-1 系统顶层数据流（测试对象边界）

1.2 术语及主要名称介绍

- **白盒测试**: 基于代码逻辑路径的测试方法，关注语句、条件、路径覆盖。
- **黑盒测试**: 基于需求输入输出的测试方法，关注功能行为与边界。
- **冒烟测试**: 以最小用例集合快速验证核心流程可用性。
- **基线路径**: 选取系统中最关键、最高频的业务路径进行持续回归。

1.3 参考文献

1. 《软件测试的艺术》
2. ISTQB 基础级测试标准
3. Postman 官方文档
4. JMeter 官方文档

2 测试需求说明

2.1 编写目的

本文档用于明确测试范围、测试策略、测试环境与验收标准，确保系统交付前在功能正确性、稳定性与可维护性方面满足课程项目要求。

2.2 系统功能需求

本轮测试覆盖以下功能域：

- 登录鉴权（管理端与小程序）。
- 商品管理（分页、增删改、图片上传）。
- 购物车与订单结算（加购、结算、订单查询、支付）。
- 库存告警（生成、分页查询、处理）。
- 摄像头与巡检调度（绑定、预览、手动触发、参数更新）。
- AI 接口联动（文本检索、中心识别、批量识别）。

2.3 非功能性需求指标

指标类别	目标值	说明
接口响应	常规接口 P95 <= 200ms	在本地联调网络下，剔除图片上传与视频流场景
识别能力	单帧推理时延可控、结果稳定返回	重点验证接口可用性与字段完整性
一致性	订单与库存同事务提交	失败时必须回滚，避免脏数据
可恢复性	异常时返回统一错误结构	全局异常处理器输出 code/message/data

表 4-1 非功能性测试指标

2.4 环境需求

- 操作系统：Windows 10/11。
- JDK：21。
- Maven：3.9+。
- Python：3.10+，依赖 fastapi/uvicorn/opencv-python/numpy/pydantic。
- Node.js：18+（项目当前环境为 Node.js 24）。
- 数据库与缓存：MySQL 8.x、Redis 8.x。
- 客户端工具：Postman、微信开发者工具。

2.5 测试人员要求

1. 熟悉 RESTful 接口测试、HTTP 状态码与 JSON 结构校验。
2. 能阅读 Java/Python 关键业务代码，完成白盒路径分析。
3. 能使用 Postman/JMeter 执行功能与性能测试。

2.6 测试标准

- 关键业务链路（登录、加购、结算、告警）必须通过。
- 高风险缺陷（导致数据错误或流程中断）必须关闭。
- 非阻断类问题可带条件上线，但需有规避方案与修复计划。

3 测试计划

本次测试采用“先冒烟后回归”的策略：

- 1. 阶段一：接口冒烟测试，验证主流程可跑通。
- 2. 阶段二：白盒路径覆盖，针对结算、鉴权、巡检等核心代码进行路径分析和用例补齐。
- 3. 阶段三：黑盒方法测试，按等价类、边界值、判定表、场景法执行回归。
- 4. 阶段四：性能与异常测试，验证可用性与容错性。

入口条件：后端、AI、数据库、缓存服务已启动；初始化数据已导入。退出条件：主流程缺陷清零，回归用例通过率满足验收要求。

4 测试过程及用例

4.1 白盒测试用例

白盒测试聚焦 `OrderServiceImpl.checkout`、`JwtAuthInterceptor.preHandle`、`CameraScanScheduler.executeScanBatchTask` 三条关键链路。

语句覆盖

用例 ID	测试目标	预期结果
WB-S-01	用户未登录直接结算	返回 401，流程中止
WB-S-02	购物车为空结算	返回 400，提示购物车为空
WB-S-03	库存充足正常结算	生成订单、写入明细、库存扣减成功
WB-S-04	AI 批量识别调用异常	返回 502，调度任务记录错误日志

表 4-2 白盒语句覆盖用例

条件覆盖

- 条件 C1: `userId == null || userId < 1`
- 条件 C2: `goods.stock < quantity`
- 条件 C3: `remainStock < 10`

对应条件覆盖用例如下：

用例 ID	输入场景	覆盖条件	预期结果
WB-C-01	无 Token 请求	C1=True	返回 401
WB-C-02	库存不足下单	C2=True	事务回滚，返回库存不足
WB-C-03	库存临界值下单	C3=True	订单成功且生成告警
WB-C-04	库存充足下单	C1/C2/C3=False	订单成功，无告警

表 4-3 白盒条件覆盖用例

基本路径覆盖

以结算模块构建控制流图后选取 4 条基本路径：

1. P1：未登录路径（入口即失败）。
2. P2：购物车为空路径（参数通过但业务拦截）。
3. P3：库存不足路径（中途失败并回滚）。
4. P4：成功路径（订单与库存均提交，异步清空购物车）。

4.2 黑盒测试用例

等价类划分

功能	有效等价类	无效等价类	判定
登录	username/password 非空且匹配	用户名空、密码空、账号不存在、密码错误	返回 200/401/400
文本检索	keyword 非空, $k \geq 1$	keyword 为空, $k < 1$	返回商品列表或 400
商品新增	name/price/stock/shelfId 合法	关键字段缺失或类型非法	返回 200 或 400

表 4-4 黑盒等价类设计

边界值测试

边界值选取：page=1、size=1、k=1、k=100、stock=0。预期：边界内可处理，越界返回参数错误或空结果。

判定表法

以“是否允许结算”为例构建判定表：

- 条件：已登录 / 购物车非空 / 库存充足。

- 动作：允许结算并生成订单 / 拒绝并给出原因。

规则结论：仅当三个条件同时满足时执行结算；任一条件不满足均返回失败。

因果图法

因：低库存触发阈值（库存 < 安全库存）果：生成告警记录，并在管理端待处理列表可见。验证点：告警类型、消息文本、状态字段是否正确。

场景法

端到端场景 S1：

1. 小程序登录。
2. 文本检索商品并加入购物车。
3. 发起结算并支付。
4. 管理端查看订单与告警变化。

S1 预期：订单生成成功；库存扣减；低库存商品触发告警。

正交实验法

针对“搜索体验”选取三因素进行组合：

- 因素 A：搜索方式（文本/图片）。
- 因素 B：返回数量 k（3/10/20）。
- 因素 C：网络状态（正常/弱网）。

通过正交组合验证搜索稳定性与可用性，减少全排列测试成本。

4.3 性能测试

本轮性能测试采用“轻量压测 + 接口基准”方式：

- 工具：Postman Runner（基准）、JMeter（并发脚本）。
- 对象：/api/goods/page、/api/order/checkout、/api/applet/search/text。
- 指标：平均响应时间、P95、错误率。

阶段性结论：当前版本已完成单用户和小并发基准验证，系统无阻断性错误；大并发压测建议在独立性能环境继续执行。

4.4 其他测试

- 异常测试：非法 JSON、错误 Content-Type、未授权 Token。
- 兼容性测试：管理端 Chrome 浏览器、小程序开发者工具。
- 恢复测试：AI 服务不可用、Redis 不可用时的降级与报错行为。
- 安全测试：鉴权拦截、越权访问、敏感错误信息屏蔽。

5 测试报告及分析

5.1 测试报告

基于本阶段用例执行与联调记录，可得结论：

- 1. 核心业务链路（登录、检索、加购、结算、告警）可闭环运行。
- 2. 后端统一异常处理机制有效，错误返回结构一致。
- 3. 事务型结算逻辑能够在库存不足场景下回滚，避免数据不一致。

5.2 缺陷报告

编号	缺陷描述	严重级别	状态
BUG-01	管理端摄像头预览 URL 存在重复前缀 (/api/api/...) 风险，可能导致预览失败	中	待修复
BUG-02	小程序登录演示环境固定 test_code，未接入真实 code2Session	中	已知限制
BUG-03	管理员密码明文存储与比对，生产环境需替换为 BCrypt	高	待修复

表 4-5 主要缺陷汇总

5.3 分析总结

- 系统在课程项目目标范围内具备可交付性，功能链路完整。
- 当前主要风险集中于“生产级安全与工程化能力”，不影响演示，但需在后续版本优先处理。
- 下一阶段建议补齐自动化测试（JUnit + 前端 E2E）与专项性能压测报告，提升回归效率与可信度。

第 5 章 系统安装部署手册

1 编写目的

本文档用于指导项目成员或实施人员在 Windows 环境完成系统部署，确保前后端、AI 服务、数据库与缓存服务能够按统一流程快速搭建并联动验证。

1.1 读者对象

- 项目实施工程师
- 系统运维人员
- 课程验收与演示准备人员

1.2 参考资料

- Spring Boot 官方文档
- FastAPI 官方文档
- 微信小程序开发者工具文档
- 项目数据库脚本：retail/retail_db.sql

2 安装部署要求

2.1 服务器要求

类型	最低配置	推荐配置
CPU	4 核	8 核及以上
内存	8GB	16GB 及以上
磁盘	20GB 可用空间	50GB SSD 可用空间
网络	千兆局域网	千兆局域网/专线
外设	USB 摄像头（可选）	高清摄像头 + 稳定光照

表 5-1 服务器硬件要求

2.2 支撑软件安装要求

软件	版本	说明
JDK	21	运行 retail-server
Maven	3.9+	构建与启动 Spring Boot
Python	3.10+	运行 retail-ai
Node.js	18+	构建/运行 retail-admin-pro
pnpm	8+	前端依赖管理
MySQL	8.x	业务数据库 retail_db
Redis	8.x	推荐流与登录缓存
微信开发者工具	最新版	运行 retail-customer

表 5-2 支撑软件要求

2.3 应用软件部署要求

- 开放端口：5173（管理端开发）、8080（后端）、8000（AI）、3306（MySQL）、6379（Redis）。
- 修改后端配置文件 retail-server/src/main/resources/application.yml 中数据库账号密码。
- 确保日志目录可写；默认日志路径为 E:/retail/log/retail-server.log，如不存在请提前创建或调整配置。

3 部署方案

3.1 环境部署

步骤一：数据库初始化

1. 创建并初始化业务库：
 - 执行脚本：retail/retail_db.sql
 - 可选测试数据：mock_data.sql
2. 默认管理员账号：admin，默认密码：123456。

步骤二：Redis 启动

可使用项目内置 Redis 目录启动（Windows）：

- 启动脚本:Redis-8.6.2-Windows-x64-msys2/Redis-8.6.2-Windows-x64-msys2/start.bat

步骤三：AI 服务部署 (retail-ai)

1. 进入目录: retail/retail-ai
2. 创建虚拟环境并安装依赖: `python -m venv .venv, pip install -r requirements.txt`
3. 启动服务: `python main.py`
4. 启动成功后默认监听: `http://localhost:8000`

步骤四：后端服务部署 (retail-server)

1. 进入目录: retail/retail-server
2. 执行启动命令: `mvn spring-boot:run`
3. 若系统未安装 Maven, 可使用项目内 Maven:
`../../apache-maven-3.9.14/bin/mvn.cmd spring-boot:run`
4. 启动后访问: `http://localhost:8080`

步骤五：管理端部署 (retail-admin-pro)

1. 进入目录: retail/retail-admin-pro
2. 安装依赖: `pnpm install`
3. 启动开发服务: `pnpm dev`
4. 访问地址: `http://localhost:5173`

步骤六：小程序端部署 (retail-customer)

1. 使用微信开发者工具导入目录: retail/retail-customer
2. 检查 `project.config.json` 的 `appid` 配置。
3. 检查 `app.js` 与 `utils/request.js` 中后端地址 (默认 `http://localhost:8080/api`)。
4. 编译运行并验证登录、搜索、结算流程。

3.2 系统部署

部署拓扑

系统采用“前端 + 后端 + AI + 数据库 + 缓存”分层部署:

- 管理端通过 Vite 代理访问后端 `/api`。
- 小程序端通过 `wx.request` 访问后端 `/api/applet/*`。
- 后端通过 HTTP 调用 AI 服务 `/api/ai/*`。
- 后端通过 JDBC 访问 MySQL, 通过 Lettuce 访问 Redis。

部署验证清单

建议按以下顺序验证：

1. 访问 `http://localhost:8080/api/auth/login` 登录接口返回正常。
2. 管理端可登录并进入“商品管理”页面。
3. 管理端可查看“视觉调度”并触发一次巡检。
4. 小程序可登录并完成“搜索-加购-结算”流程。
5. 数据库中 `sys_order`、`sys_order_item` 有新增记录。

常见故障与处理

- **后端启动失败（数据库连接）：**检查 `application.yml` 的 `datasource` 配置与 MySQL 服务状态。
- **管理端接口 401：**检查是否携带 Token，确认登录后本地缓存是否写入。
- **AI 调用 502：**检查 `retail-ai` 是否在 8000 端口运行。
- **摄像头不可用：**检查摄像头权限、设备占用状态及 DirectShow 驱动。

第 6 章 系统用户操作手册

1 商超管理员角色操作手册

1.1 登录功能使用说明

1. 在浏览器访问管理端首页（默认：`http://localhost:5173`）。
2. 输入管理员账号与密码（默认 `admin/123456`）。
3. 点击“登录”进入系统首页。

异常提示：若提示“未登录或 Token 无效”，请重新登录并确认后端服务正常运行。

1.2 商品管理功能使用说明

进入菜单“商品管理”后，可进行以下操作：

1. **查询商品**：输入商品名称关键字，点击“搜索”。
2. **新增商品**：点击“新增商品”，填写名称、价格、库存、货架号并上传图片后提交。
3. **编辑商品**：点击列表“编辑”按钮更新商品信息。
4. **删除商品**：点击“删除”并确认。

操作结果：系统通过接口 `/api/goods`、`/api/upload` 返回成功消息，列表自动刷新。

1.3 视觉调度功能使用说明

进入菜单“视觉调度”后，可进行以下操作：

1. **新增摄像头绑定**：选择物理摄像头索引并填写货架号。
2. **编辑绑定关系**：修改指定摄像头对应货架号。
3. **删除摄像头**：移除无效或停用设备。
4. **更新巡检参数**：设置巡检间隔（分钟）和分批大小（台）。
5. **手动触发巡检**：点击按钮立即执行一次全量扫描。

建议：修改巡检参数后观察日志，确认任务已按新参数重启。

1.4 告警与看板功能使用说明

1. 在告警列表中查看未处理库存预警。

2. 对已确认问题执行“处理告警”操作。
3. 在数据看板查看销售趋势、库存健康度与近期告警概览。

2 顾客角色操作手册

2.1 登录功能使用说明

1. 打开小程序进入“我的”页面。
2. 点击“登录”，授权微信用户信息。
3. 登录成功后，系统在本地保存 Token 与用户资料。

说明：演示环境中小程序登录使用固定测试码 `test_code`。

2.2 查询功能使用说明

1. 进入“查询”页输入关键词。
2. 系统显示搜索结果及推荐商品。
3. 点击商品可查看详情，或直接加入购物车。

提示：若返回“未找到相关商品”，可尝试更换关键词或使用图片识别。

2.3 视觉识别寻物功能使用说明

1. 进入“识别”页，选择“拍照”或“从相册选择”。
2. 系统上传图片并执行 AI 识别。
3. 识别结果展示商品名称、价格，点击后可加入购物车。

异常处理：若提示“识别失败”，请检查网络和图片清晰度后重试。

2.4 购物车与结算功能使用说明

1. 在“购物车”页勾选商品并调整数量。
2. 点击“去结算”进入订单确认页。
3. 确认金额后提交结算，系统生成订单号。

结算规则：库存不足时系统会阻止下单，并提示库存异常信息。

2.5 订单、支付与充值功能使用说明

1. 在“我的-订单”中查看历史订单与状态。
2. 对待付款订单进入支付页，选择余额或微信方式。

3. 余额不足时可进入充值页完成账户充值。

3 系统后台角色操作手册

3.1 巡检任务运行说明

系统后台内置摄像头巡检任务，默认每 5 分钟执行一次：

- 读取状态正常的摄像头列表。
- 分批调用 AI 批量识别接口。
- 同步识别到的商品与货架关系。
- 记录任务日志并返回结果。

3.2 参数调整与手动触发说明

运维人员可在管理端“视觉调度”页面完成：

1. 调整巡检间隔与批次大小。
2. 手动触发全量巡检，用于快速复核库存陈列。

3.3 异常处理说明

- **AI 服务不可达：**后端返回 502，建议先恢复 retail-ai 后重试。
- **数据库异常：**结算流程自动回滚，避免脏数据。
- **摄像头占用：**释放视频流资源后重新预览。
- **Token 过期：**前端清理登录态并引导重新登录。

3.4 运维建议

1. 每日检查 retail-server.log 和巡检任务日志。
2. 每周备份 MySQL 业务库和关键配置文件。
3. 定期清理无效摄像头绑定与历史无效告警。